

Regularization and Initialization

2026 Winter DL Session 4주차



고려대학교
KOREA UNIVERSITY

KUBIG
DATA SCIENCE & AI

Copyright © 2026
by KUBIG

목차

KUBIG 2026 Winter DL Session

01 들어가기 전에,,

02 Regularization

03 Initialization

04 CNN 맛보기!

05 다음 주차 예고

01 들어가기 전에,,

01. 들어가기 전에,,

3주차 과제 요약

- Multilayer Perceptron의 전체 로직을 구현해보자!

1. Sigmoid함수에 대한 이해와, 순전파 & 역전파 과정을 숙지
2. MLP의 구성요소인 layer, activation, optimizer를 구현
3. 모델의 hyperparameter와 optimizer를 바꾸면서 Training 결과를 확인

01. 들어가기 전에,,

3주차 과제 우수자

22기 조해진 님

Sigmoid function 이론

23기 강민혁 님
23기 하솔미 님

코드 과제

모두 과제하느라 수고많으셨습니다 ! :)

01. 들어가기 전에,,

4주차 학습 목표

정규화(Regularization)와 초기화(Initialization)를 통해
원활하고 좋은 학습 성능을 보이기 위한 방법들이 무엇인지 알아보자!

+ 이미지 데이터의 특성과 MLP 적용의 한계를 알아보자!

02 Regularization

02. Regularization

Regularization ?

Normalizaion?

Standardization?

02. Regularization

Regularization

- overfitting을 막기 위해, model의 parameter의 값을 제한시키는 방식
- simpler model을 만들기 위해 small weight를 줘 overfitting을 줄임
- cost term을 주거나 parameter 범위에 penalty를 주는 형식

02. Regularization

Regularization vs Normalization

Regularization	Normalization
• overfitting 방지 목적 • cost, penalty 부여를 통해 weight 제약을 주는 것	• 0 ~ 1로 scale 변화 (scaling 목적) • ML: normalization은 값에 의한 dominated 현상 방지 • DL : local minima에 빠지는 것을 방지해 학습 속도 향상

02. Regularization

참고. Normalization vs Standardization

- Normalization and Standardization are similar and both relate to the issue of feature scaling (to make training less sensitive to the scale of features of input data).
- If we train an algorithm using various scaled features of the input variables, then the results might be dominated by features with large magnitude.

Normalization (into [0, 1])

$$x'_i = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$



Standardization (z-score)

$$x'_i = \frac{x_i - \mu}{\sigma}$$

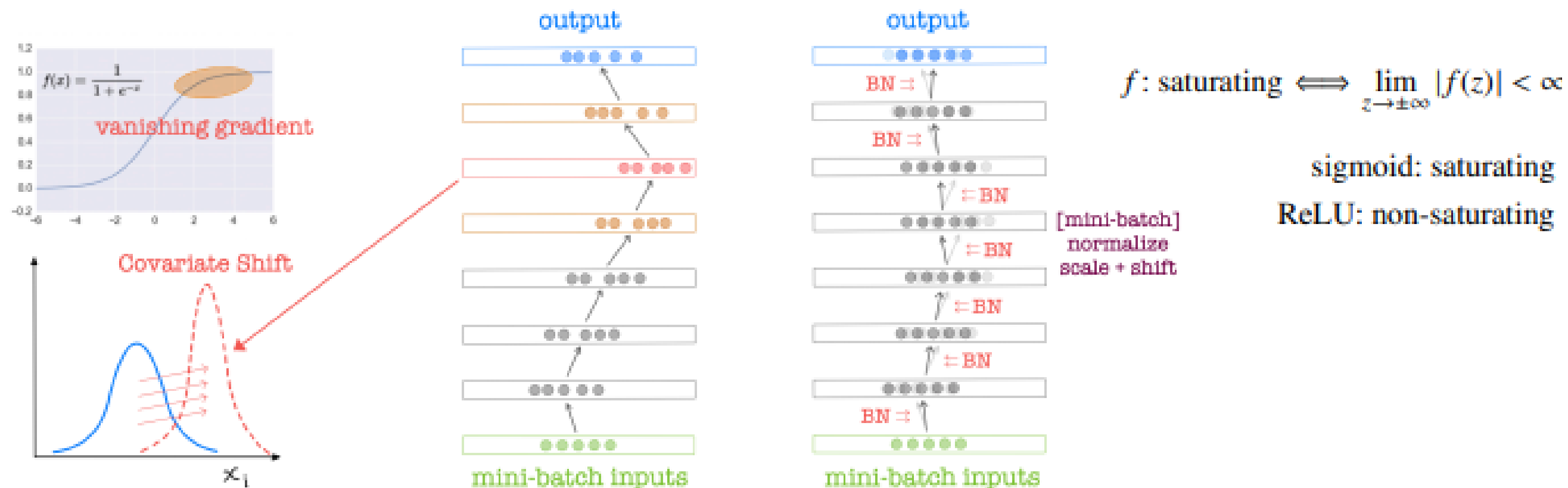


02. Regularization

Batch Normalization

Internal Covariate Shift: distribution change of each layer's inputs during training due to changes in the parameters of the previous layer. (the more layers, the more amplification)

- Since the inputs to each layer are affected by the parameters of all preceding layers, small changes to the parameters amplify the input values of the following layers.
- This requires lower learning rates and careful weight initialization, which slow down training, and makes it notoriously hard to train models with saturating nonlinearities.



02. Regularization

Batch Normalization

Batch normalization (BN) reduces Internal Covariate Shift to train fast by allowing us to use higher learning rates, saturating nonlinearities, and less careful weight initialization.

- BN transform can be freely added to any subset of activations $\{x_i^{(k)}\}$ to be normalized.
- For each activation $x_i^{(k)}$, it adds two learnable parameters scale $\gamma^{(k)}$ and shift $\beta^{(k)}$.
- $y_i^{(k)} = \text{BN}_{\gamma, \beta}(x_i^{(k)})$ depends on $\{x_1^{(k)}, \dots, x_m^{(k)}\}$ in the minibatch to be normalized.

Input: Values of x over a minibatch: $\mathcal{B} = \{x_1, \dots, x_m\}$;
Parameters to be learned: γ, β

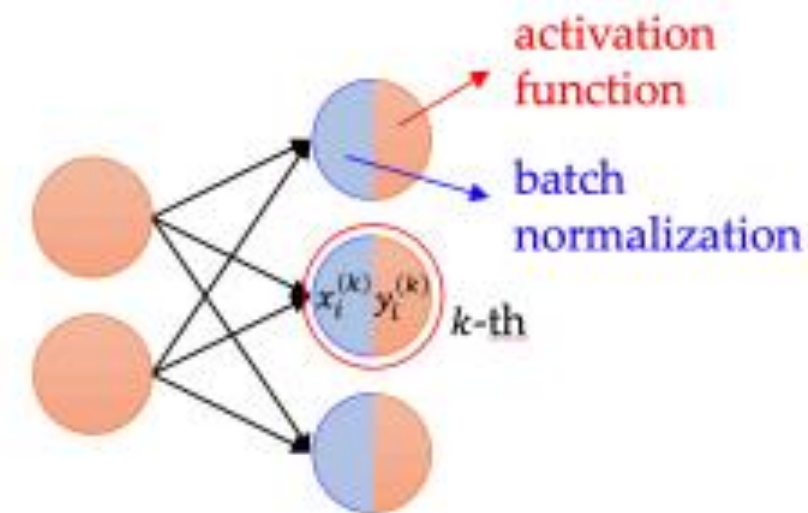
Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$ // $x_i = x_i^{(k)}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{minibatch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{minibatch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{normalize}$$

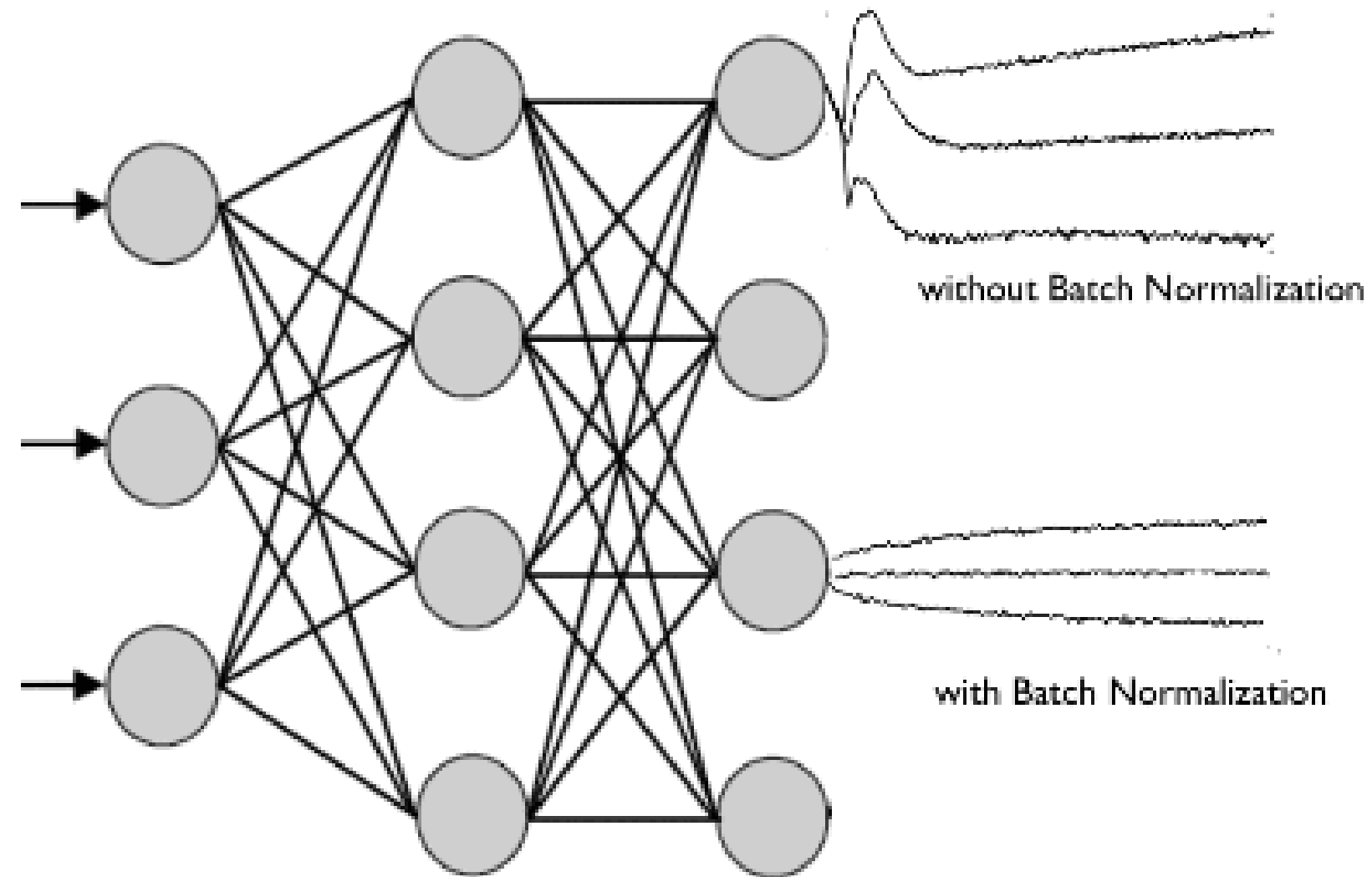
$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{scale and shift}$$



Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift, Google 2015

02. Regularization

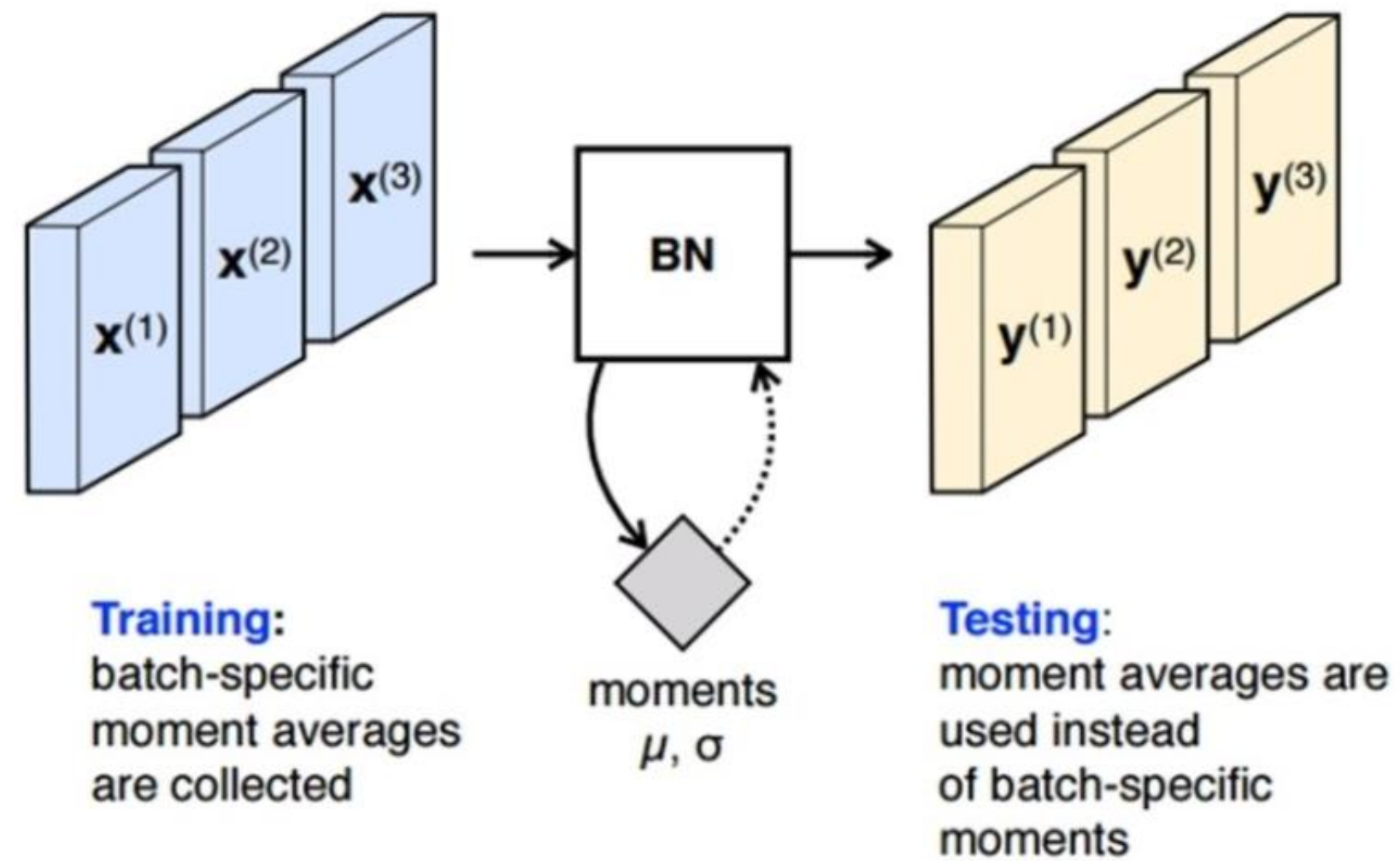
Batch Normalization



출처 : <https://eehoeskrap.tistory.com/430>

02. Regularization

Batch Normalization



주의!!!

BN은 Inference(test-time)
에 사용할 수 없음!

출처 : <https://eehoeskrap.tistory.com/430>

02. Regularization

Regularization

- Weight decay

regularization 목적(가중치 감소)도 있지만,
gradient 계산 시 계산값이 튀는 현상을 완화시켜,
안정성 보장 목적으로 더 많이 사용됨

L1 regularization (L1 norm, Lasso)

$$C_{L1} = \frac{1}{2N} \sum_{n=1}^N (\hat{y}^n - y^n)^2 + \frac{\lambda}{N} \sum_{i=1}^K |w_i|$$

L2 regularization (Weight decay, L2 norm, Ridge)

$$C_{L2} = \frac{1}{2N} \sum_{n=1}^N (\hat{y}^n - y^n)^2 + \frac{\lambda}{2N} \sum_{i=1}^K w_i^2$$

$$w_i(t+1) = w_i(t) - \eta \frac{\partial C_{L2}}{\partial w_i} = \left(1 - \frac{\eta \lambda}{N}\right) w_i(t) - \eta \frac{\partial C}{\partial w_i}$$

weight decay

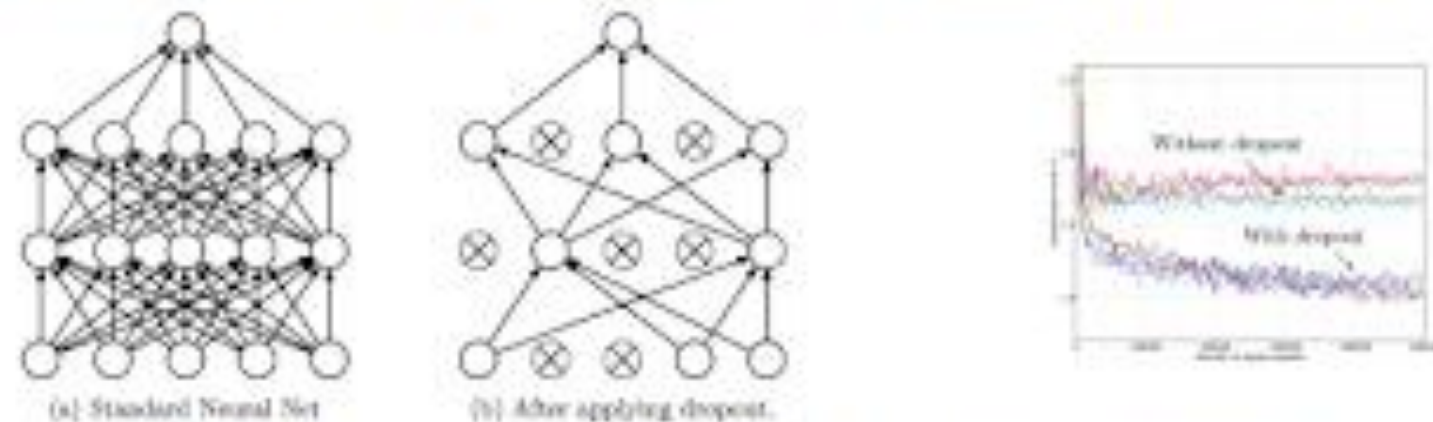
02. Regularization

Regularization

◦ Dropout

input layer와 hidden layer 사이의 노드
중 일정 비율을 제거하고 학습시키는 방법.
앙상블과 유사한 결과를 만들어냄

- Dropout is a regularization technique for reducing overfitting in neural networks by preventing nodes from co-adapting too much on training data, introduced by Hinton 2012.
- The gradient of each weight tells how it should change to reduce the cost, taking into account what all nodes are doing. So, nodes may change in a way that they fix up the mistakes of other nodes. Co-adaptation refers to when different hidden nodes have such highly correlated behavior. It is better to learn a general representation if nodes can detect features independently of each other.
- Randomly drop hidden nodes (along with their connections) in an iteration for each minibatch so that each node is omitted with probability p (independent of others) during only training. The omitted nodes and weights on this iteration are not updated during backpropagation.



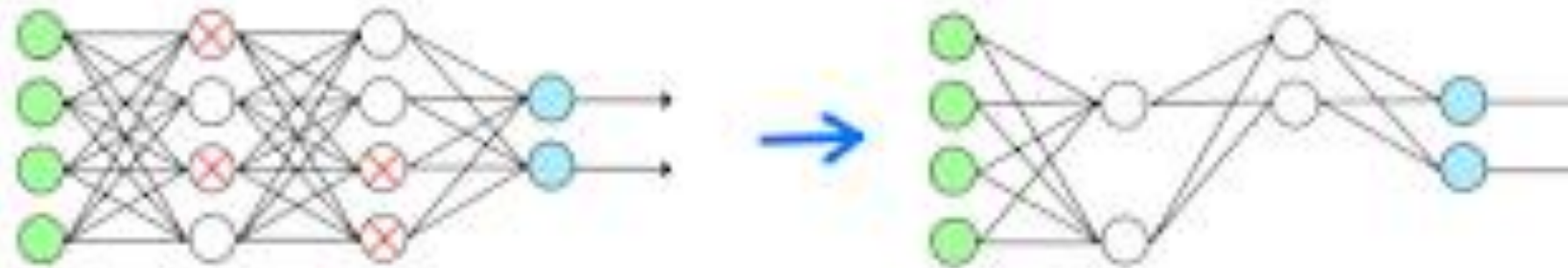
Source: Dropout: A Simple Way to Prevent Neural Networks from Overfitting by Hinton et al

02. Regularization

Regularization

Training

- For a minibatch, each node has **dropout probability p** (i.e. keeping probability $1-p$) so that we sample one from 2^N different **thinned networks**.
- Forward pass and backpropagation (gradient computing and weight update) are done only on this **thinned network**, ignoring the omitted nodes and their related weights.



Test

- For test data, we do **not apply dropout**, but take all the network weights multiplied by $1-p$ so that **this un-thinned network** with smaller weights approximates the average of the outputs of all these **thinned networks**.



02. Regularization

Regularization

- Data Augmentation

data를 증강시키는 방법

데이터 수가 적을 때, 활용하며, 잘 증강시키는 것이 key point

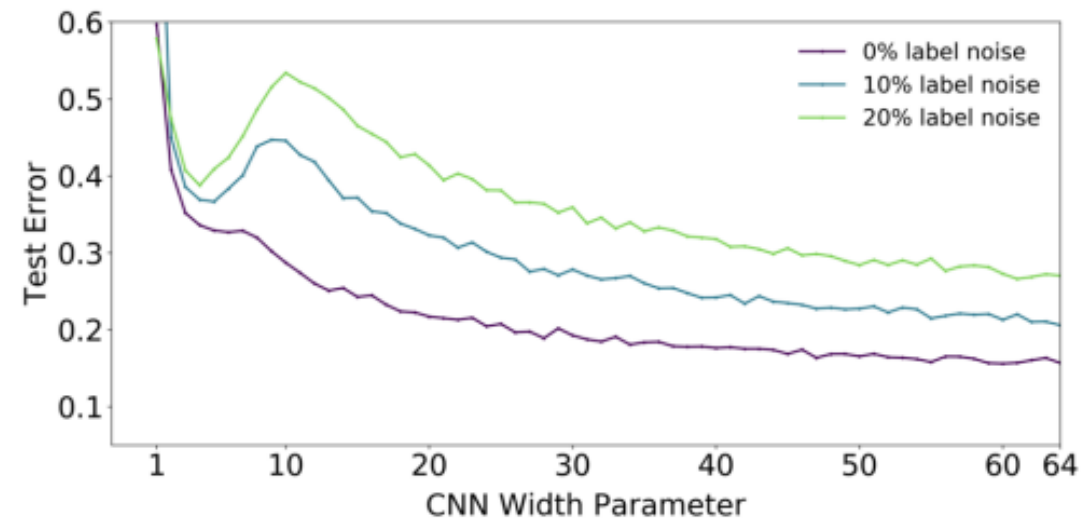
- Early Stopping

매 epoch마다 validation set에 대한 loss를 측정해 모델 종료 시점을 제어

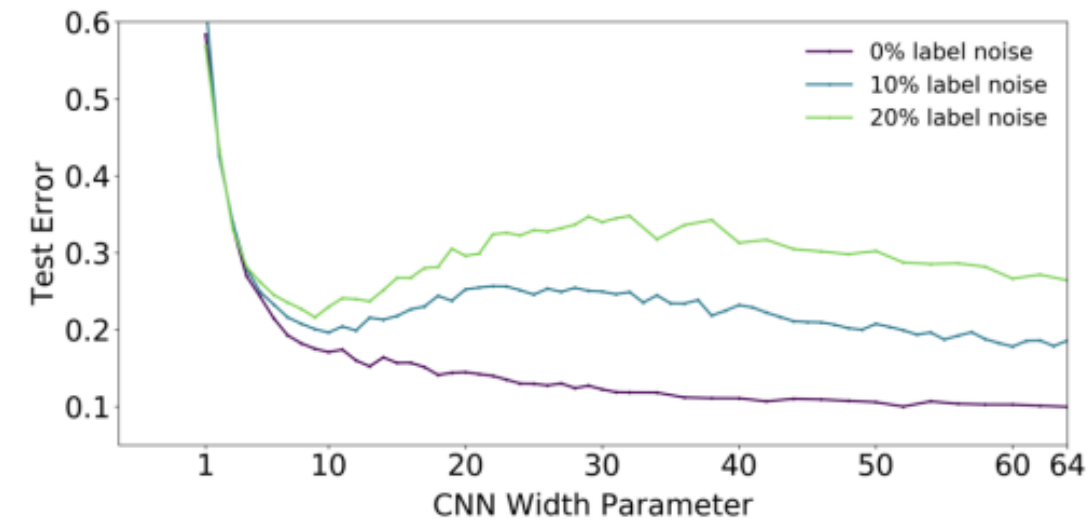
학습 stop을 통해 과적합 방지

02. Regularization

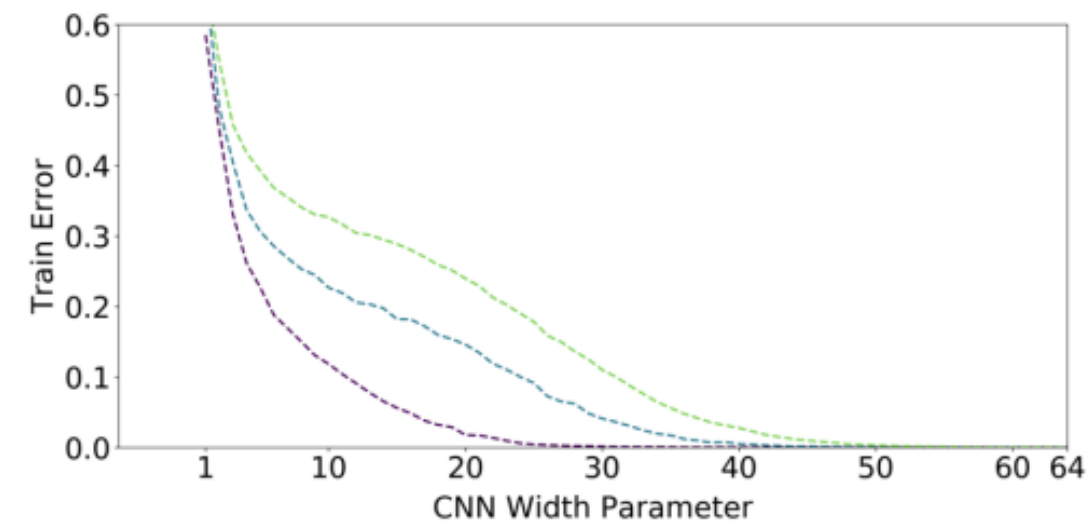
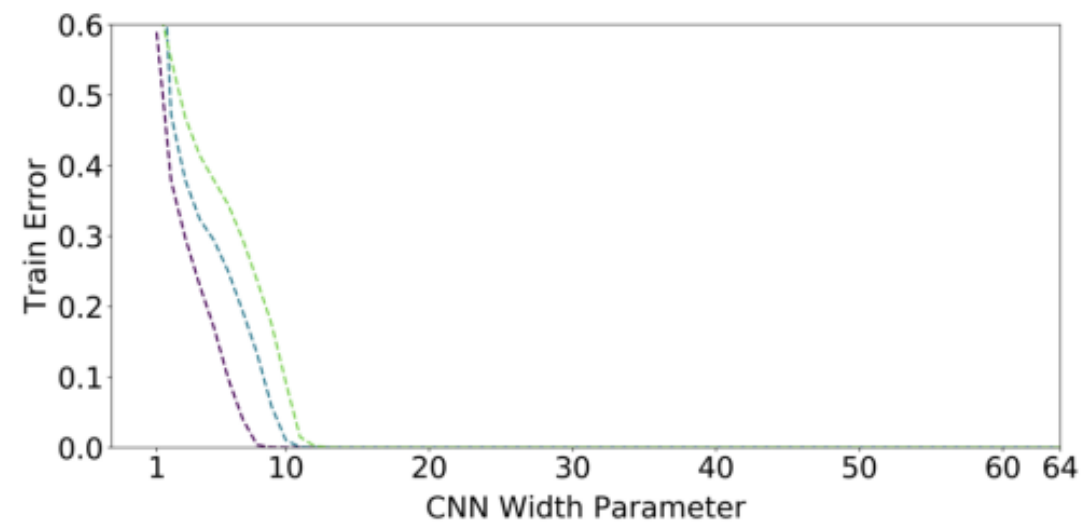
Regularization



(a) Without data augmentation.

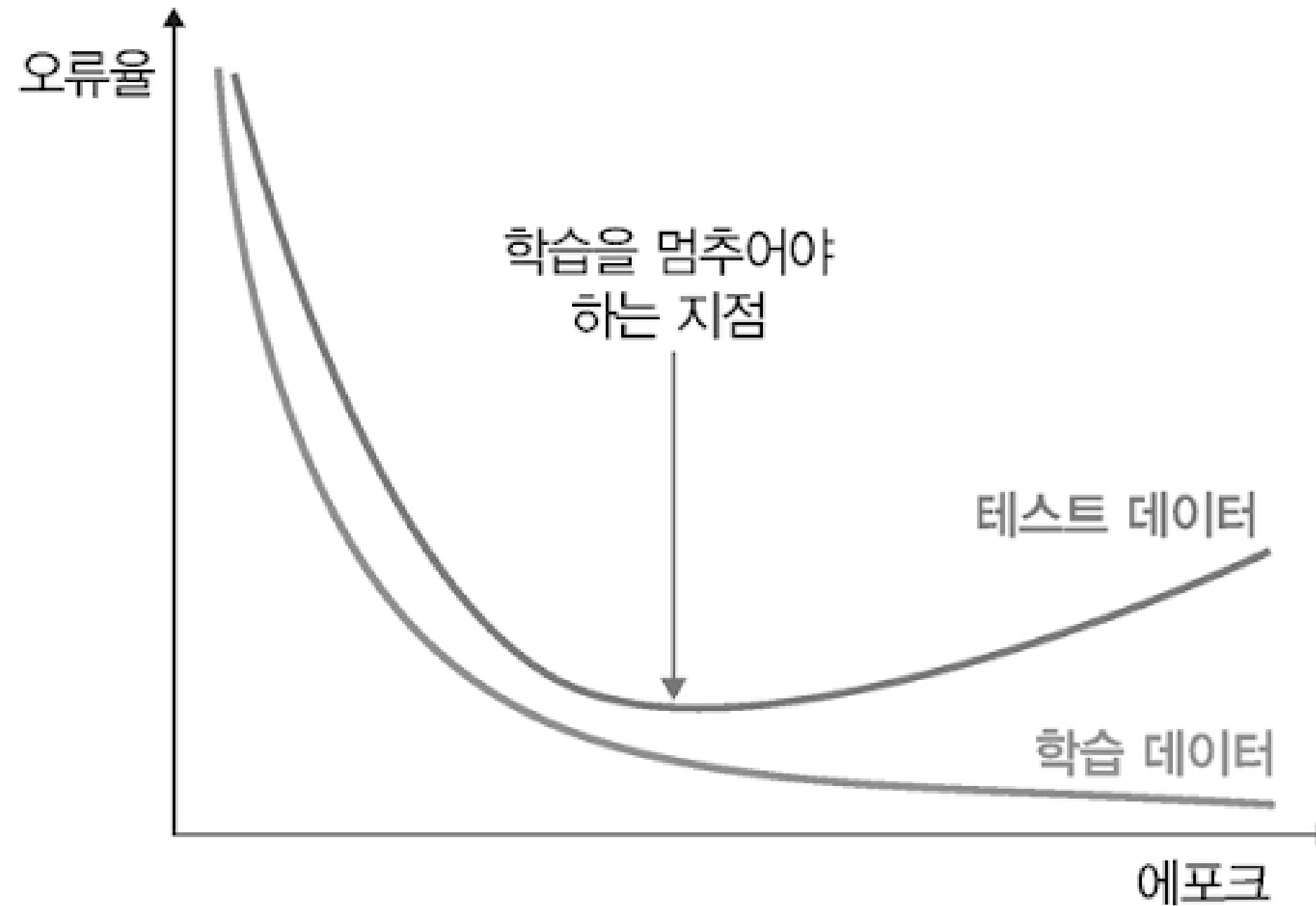


(b) With data augmentation.



02. Regularization

Regularization



02. Regularization

Regularization

- Ensemble
 - Voting
 - 대표적으로 사용하는 기법
 - 여러 분류기가 투표를 통해 최종 예측 결과 결정
 - 다른 알고리즘 가진 분류기 -> 같은 데이터셋 기반 학습 + 결합
 - Bagging
 - voting과 유사하나, training set에서 중복을 허용해 sampling, 동일 알고리즘 사용

02. Regularization

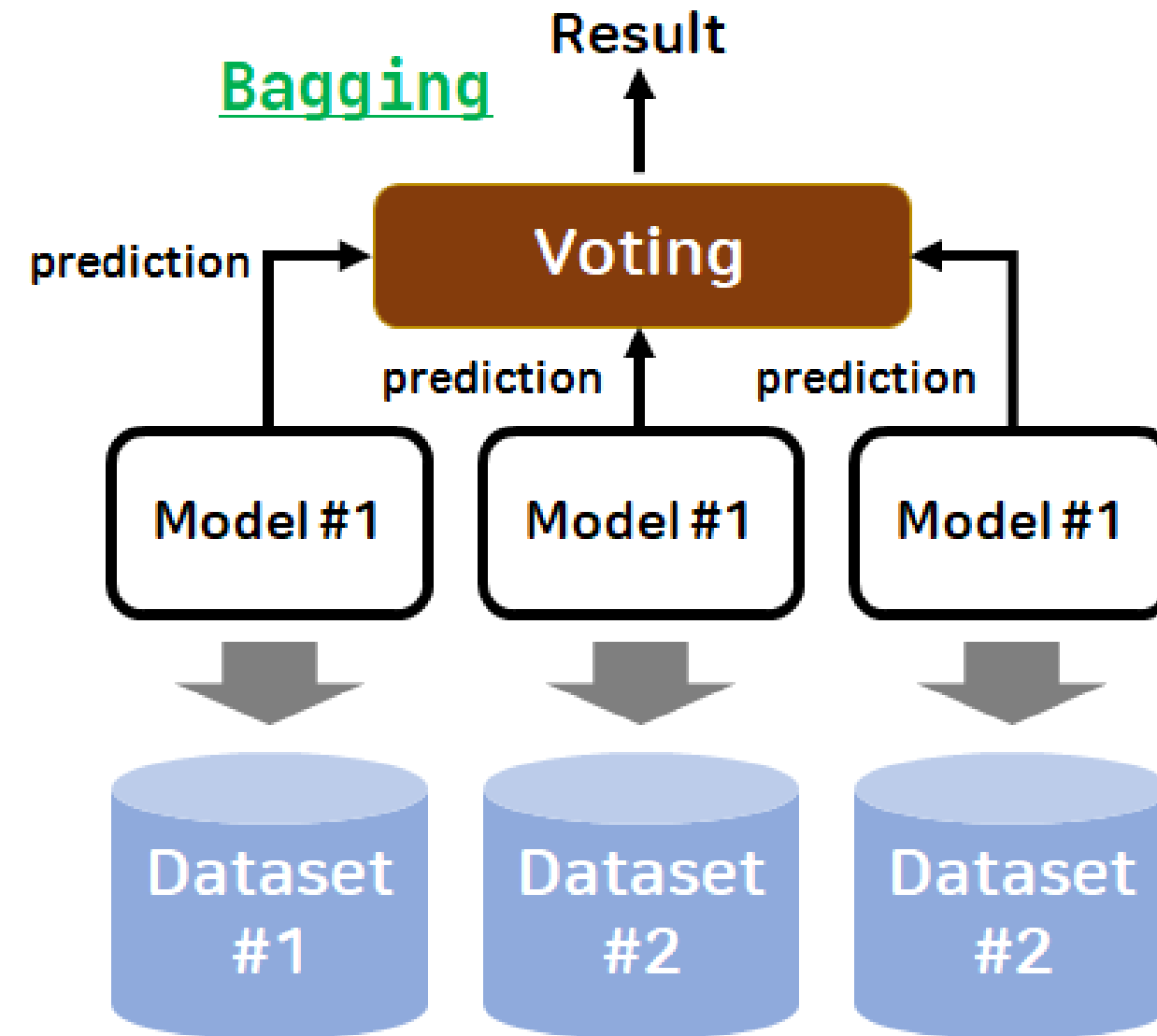
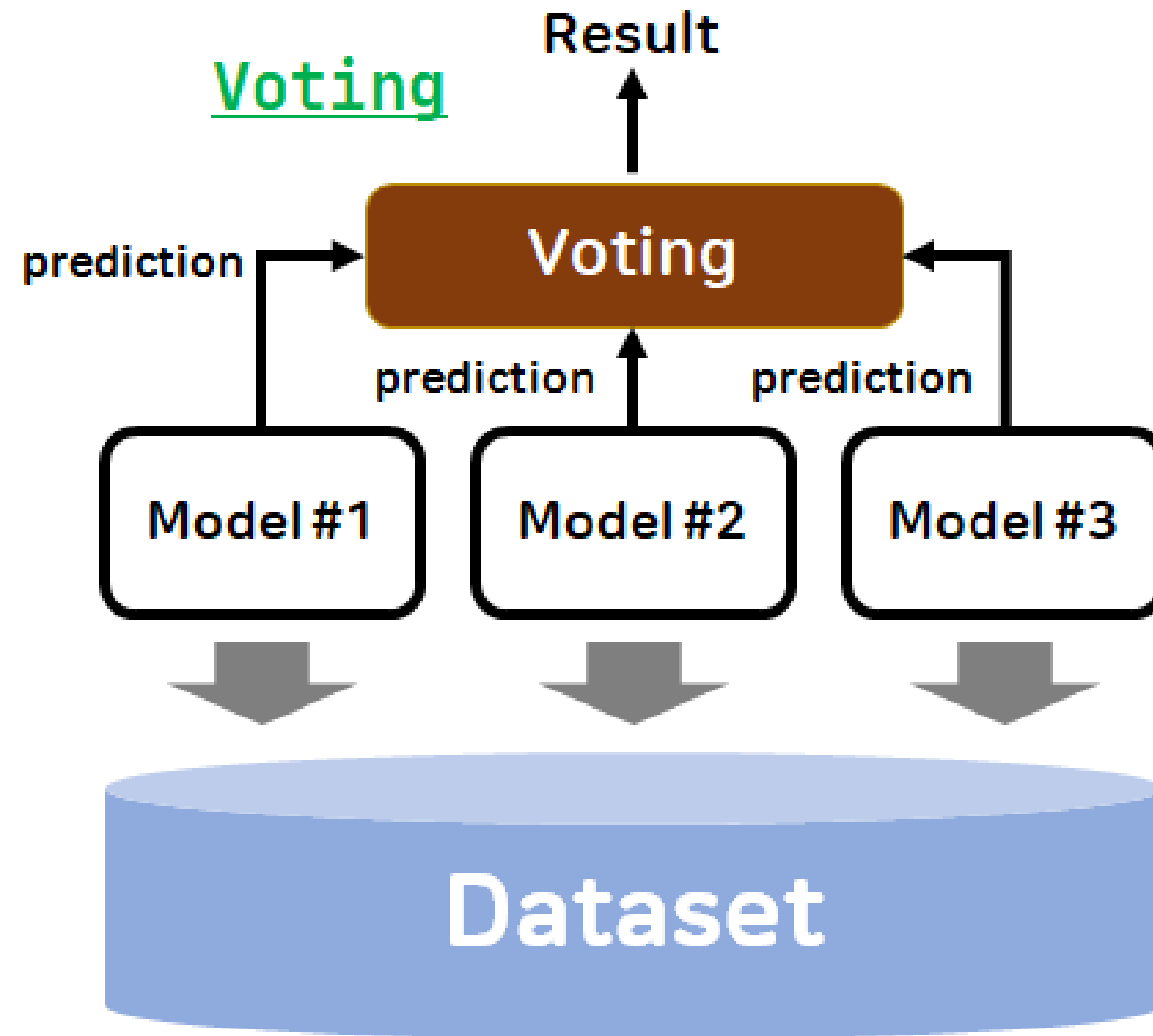
Regularization

- Ensemble
 - Boosting
 - cv, 자연어 처리에 널리 사용됨
 - 여러 약한 model 결합해 강력한 모델을 형성하는 반복 알고리즘
 - Neural Network 성능 향상에 사용
 - Stacking
 - 여러 모델을 학습시키고 결합시켜 단일 모델 성능 향상
 - 동일 training set에 대해 서로 다른 model을 학습 -> output결합
 - 개별 알고리즘 예측 데이터 기반 예측 수행

02. Regularization

Regularization

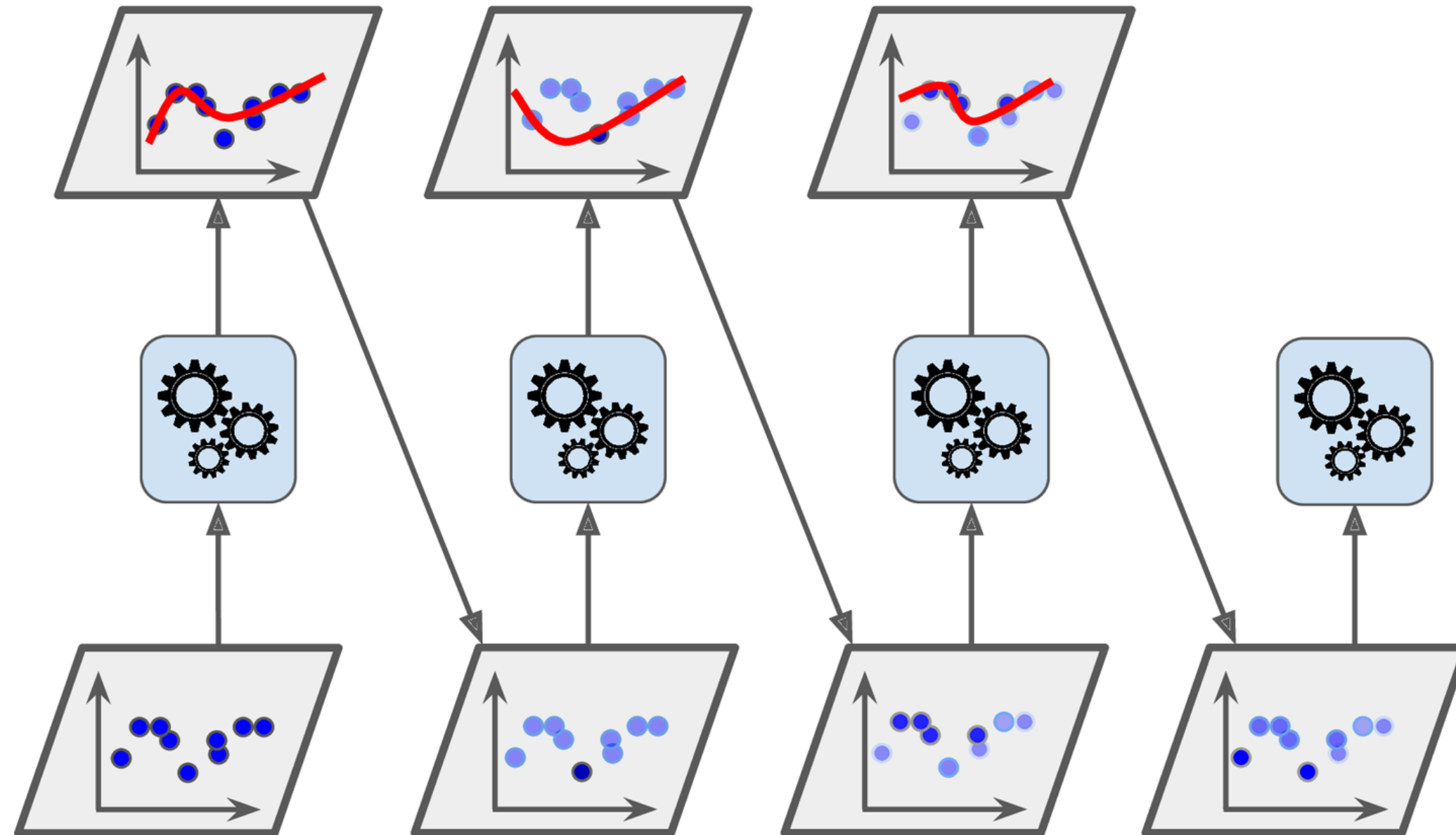
- Ensemble



02. Regularization

Regularization

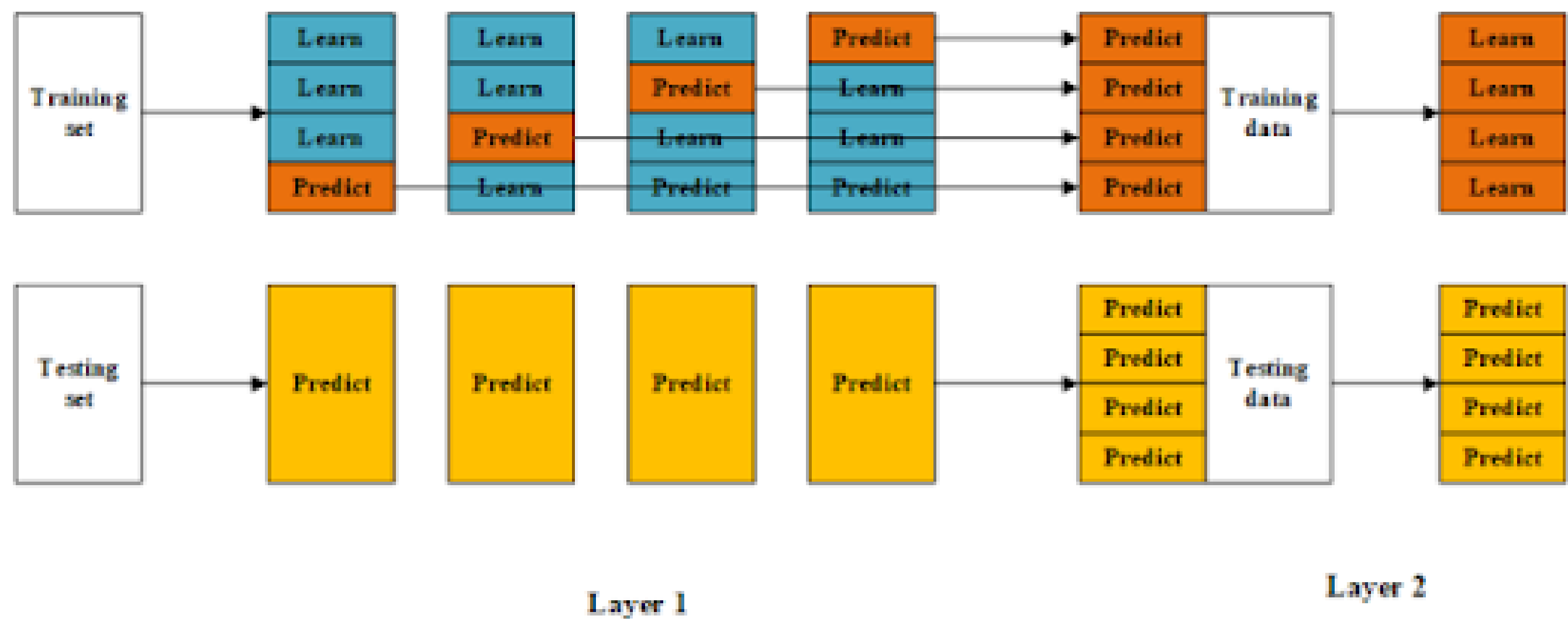
- Ensemble



02. Regularization

Regularization

- Ensemble



03 Initialization

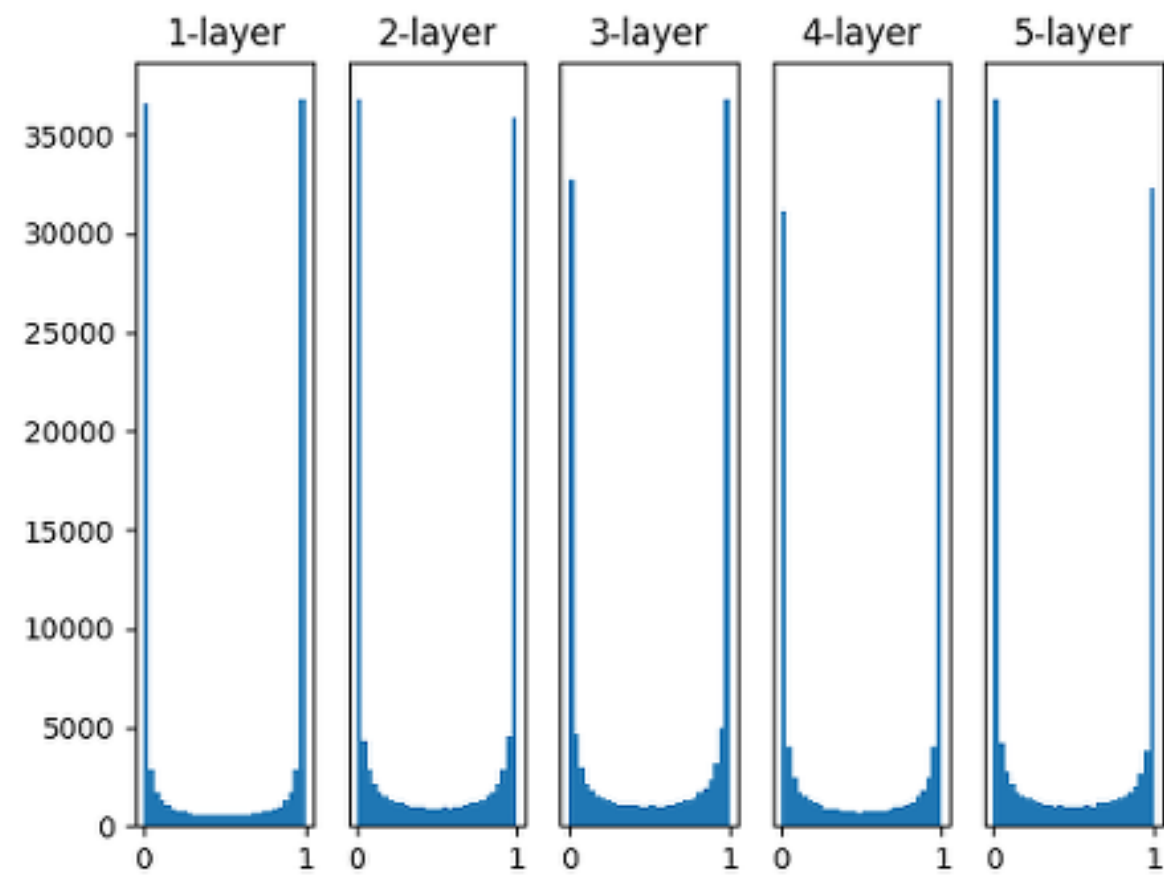
03. Initialization

Initialization

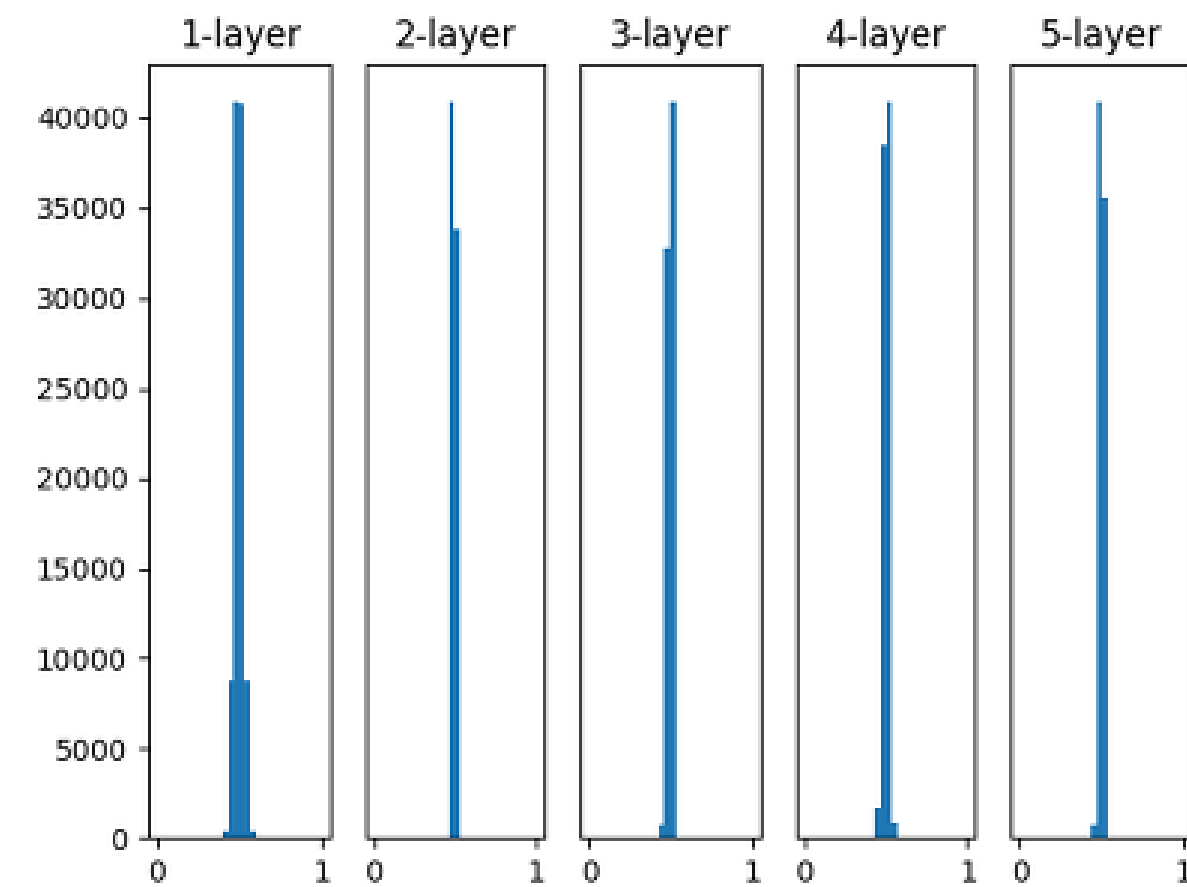
- Zero Initialization (= Same Initialization)
 - 가중치 update가 대칭적으로 일어나도록 해 모델 성능 현저히 악화
- Random Initialization
 - 지나치게 큰 분산은 활성화 값들을 0 or 1로 만들게 됨
 - 지나치게 작은 분산은 모든 활성화 값들을 유사한 값으로 만들게 됨
-> 모델 다양성 악화

03. Initialization

Initialization



Gaussian distribution + sigmoid



+ Using lower σ

Gradient Vanishing...!

03. Initialization

Advanced initialization

$$\frac{d\mathcal{L}}{dW^{(1)}} = \frac{dz^{(1)}}{dW^{(1)}} \frac{da^{(1)}}{dz^{(1)}} \frac{dz^{(2)}}{da^{(1)}} \frac{d\mathcal{L}}{dz^{(2)}}$$

If we multiply many many numbers together, what will we get?

If most of the numbers are < 1 , we get 0

If most of the numbers are > 1 , we get infinity

We only get a reasonable answer if the numbers are all close to 1!

$$\frac{d\mathcal{L}}{dW^{(1)}} = J_1 J_2 J_3 \dots J_n \frac{d\mathcal{L}}{dz^{(n)}}$$

for each $W^{(i)}$, we can write: $W^{(i)} = U^{(i)} \Lambda^{(i)} V^{(i)}$ e.g., using singular value decomposition

$W^{(i)} \leftarrow U^{(i)} V^{(i)}$ just need to force this to be identity matrix

even simpler:

```
a = get_rng().normal(0.0, 1.0, flat_shape)  ← arbitrary random matrix (doesn't really matter how)
u, _, v = np.linalg.svd(a, full_matrices=False)
# pick the one with the correct shape
q = u if u.shape == flat_shape else v  ← needed if non-square
```

guaranteed orthonormal

03. Initialization

The diagram illustrates the SVD decomposition of matrix $\mathbf{X}$. It shows a large gray box on the left containing $\mathbf{X}_{(n, p)}$. This is followed by an equals sign. To the right of the equals sign are three boxes: a blue box containing $\mathbf{U}_{(n, p)}$, a red box containing $\mathbf{D}_{(p, p)}$, and an orange box containing $\mathbf{V}^T_{(p, p)}$.

$$\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^T$$

where:

- $\mathbf{U}$ is a $n \times p$ column *orthonormal* matrix containing the **left singular vectors**.
- $\mathbf{D}$ is a $p \times p$ *diagonal* matrix containing the **singular values** of $\mathbf{X}$.
- $\mathbf{V}$ is a $p \times p$ column *orthonormal* matrix containing the **right singular vectors**.

03. Initialization

Initialization 종류

- LeCun Normal Initialization

$$W \sim N\left(0, \sqrt{\frac{1}{n_{\text{in}}}}\right)^2$$

- Xavier Normal Initialization

$$W \sim N\left(0, \sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}}\right)^2$$

- He Normal Initialization

$$W \sim N\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right)^2$$

- n_{in} node number of the previous layer, and n_{out} of the next layer.

LeCun Uniform Initialization

$$W \sim U\left[-\sqrt{\frac{1}{n_{\text{in}}}}, \sqrt{\frac{1}{n_{\text{in}}}}\right]$$

Xavier Uniform Initialization

$$W \sim U\left[-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right]$$

He Uniform Initialization

$$W \sim U\left[-\sqrt{\frac{6}{n_{\text{in}}}}, \sqrt{\frac{6}{n_{\text{in}}}}\right]$$

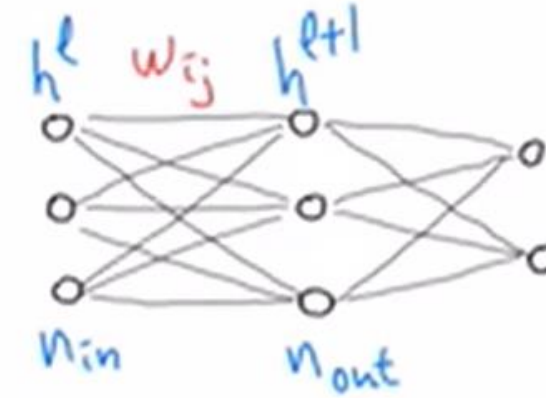
03. Initialization

Xavier Normal Initialization

- We want to initialize weights satisfying following conditions:

$$\text{Var}[h^l] = \text{Var}[h^{l+1}] \text{ 'forward'}$$

$$\text{Var}\left[\frac{\partial \text{Cost}}{\partial h^l}\right] = \text{Var}\left[\frac{\partial \text{Cost}}{\partial h^{l+1}}\right] \text{ 'backward'}$$



- Assumptions in each layer l
 - Linear Regime: activation function $f(x) = x$ for near $x=0$, and so $f'(0) = 1$.
 - All weights w_{ij}^l are independent and $\text{Var}[w_{ij}^l] = \text{Var}[w^l]$.
 - All elements of $\mathbf{h}^l = (h_1^l, \dots, h_{n_l}^l)^T$ are independent and $\text{Var}[h_i^l] = \text{Var}[h^l]$.
 - All weights and elements of an activation vector are independent of each other.
 - Each of their distributions is symmetric around 0, and so $E[w_{ij}^l] = E[h_i^l] = 0$

03. Initialization

Xavier Normal Initialization

- Forward Propagation satisfying $\text{Var}[h^l] = \text{Var}[h^{l+1}]$

$$h_j^{l+1} = \sum_{i=1}^{n_l} w_{ij}^l h_i^l \Rightarrow \text{Var}[h^{l+1}] = n_l \text{Var}[w^l h^l] = n_l \text{Var}[w^l] \text{Var}[h^l] \Rightarrow n_l \text{Var}[w^l] = 1$$

Note that $\text{Var}[XY] = E[X]^2 \text{Var}[Y] + E[Y]^2 \text{Var}[X] + \text{Var}[X] \text{Var}[Y]$ and $E[w_{ij}^l] = E[h_i^l] = 0$.

- Backward Propagation satisfying $\text{Var}\left[\frac{\partial \text{Cost}}{\partial h^l}\right] = \text{Var}\left[\frac{\partial \text{Cost}}{\partial h^{l+1}}\right]$

$$\frac{\partial \text{Cost}}{\partial h_i^l} = \sum_{j=1}^{n_{l+1}} w_{ij}^l f'(\cdot) \frac{\partial \text{Cost}}{\partial h_j^{l+1}} \Rightarrow \text{Var}\left[\frac{\partial \text{Cost}}{\partial h^l}\right] = n_{l+1} \text{Var}[w^l] \text{Var}\left[\frac{\partial \text{Cost}}{\partial h^{l+1}}\right] \Rightarrow n_{l+1} \text{Var}[w^l] = 1$$

Note that $f'(0) = 1$ and $E[w_{ij}^l] = E\left[\frac{\partial \text{Cost}}{\partial h_j^{l+1}}\right] = 0$.

- Thus, in each layer l , we initialize weights w_{ij}^l satisfying $\text{Var}[w_{ij}^l] = \frac{2}{n_l + n_{l+1}} = \frac{2}{n_{\text{in}} + n_{\text{out}}}$.

04

CNN 맛보기!

04. CNN 맛보기!

Convolutional Neural Network(CNN)

Image Recognition Task에 사용
특히, Image Classification에 많이 쓰임.

CNN의 가장 큰 특징

1. Weight Sharing by Convolution

Input image 사이즈가 너무 큰 경우 weight 수가 너무
많이 필요함. Input dimension이 큰 경우 weight
sharing을 하면 weight의 수를 어느정도 고정시킬 수 있음.

2. Solve Translation invariance by Max pooling

그림을 이동시켜도 실제로는 같은 그림.
하지만 컴퓨터 데이터는 픽셀별로 데이터를 인식하기에, 회전이나 변형이
조금만 가해져도 컴퓨터는 전혀 다른 이미지로 해석

04. CNN 맛보기!

Image data on Computers

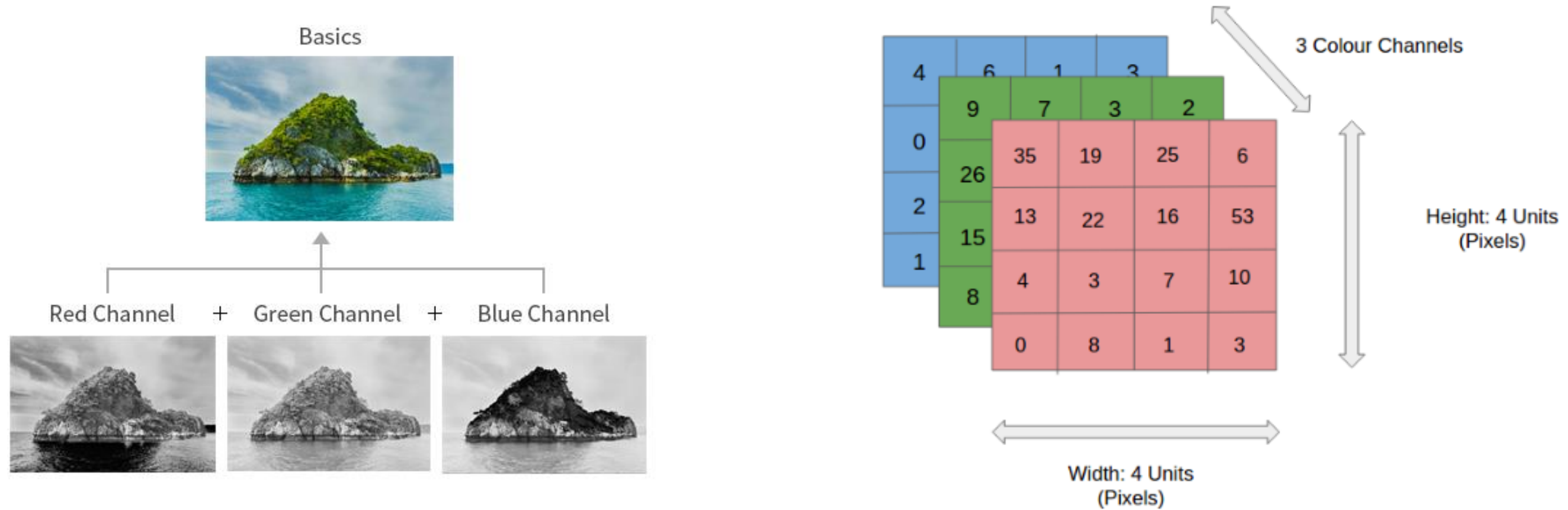
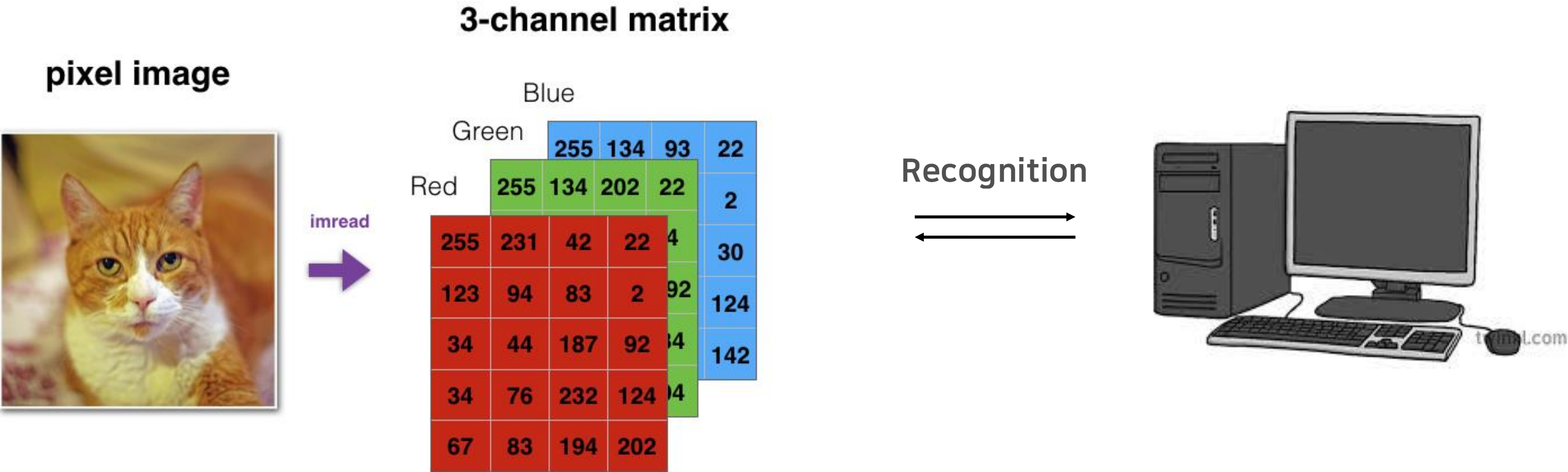


Image data는 각 pixel당 3개의 RGB값을 가지고 있음.

- 이미지 텐서의 크기: $H * W * C$
- #(Channel) = 3 : RGB 채널 이미지
- #(Channel) = 1 : 흑백.이미지

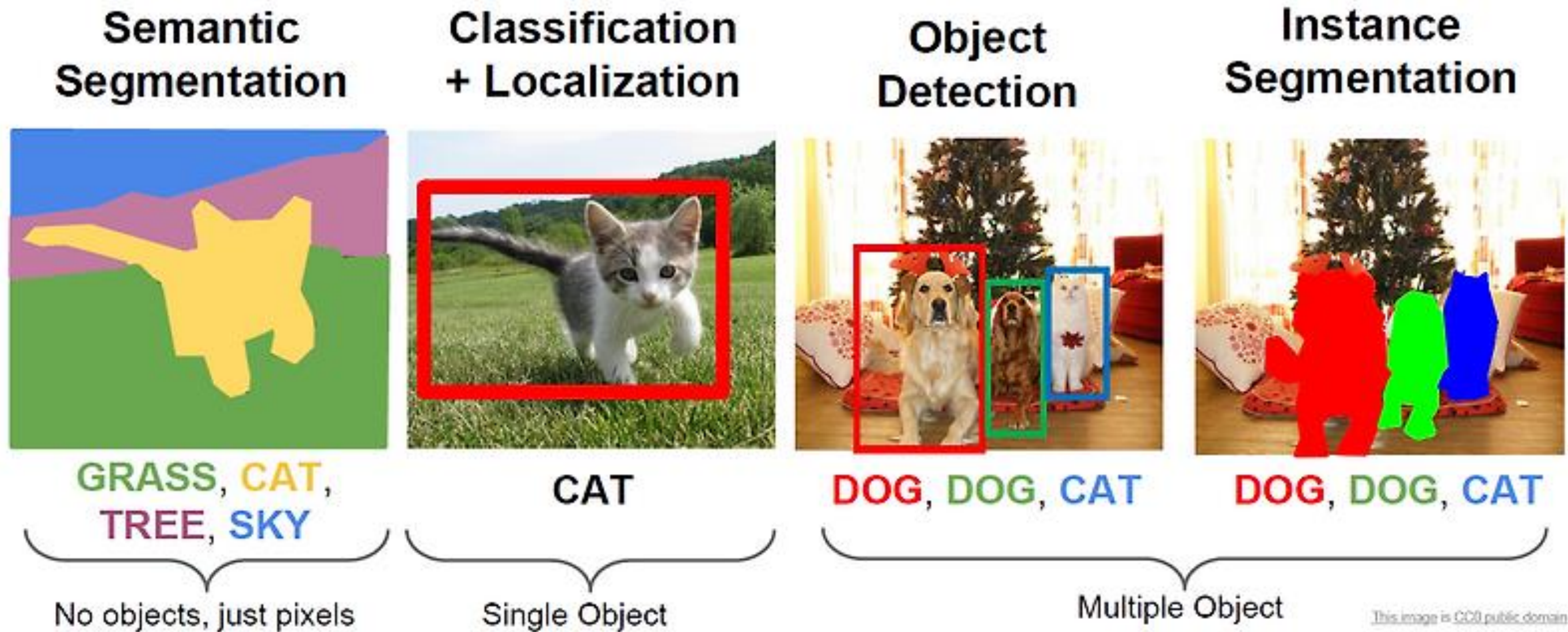
04. CNN 맛보기!

Image data on Computers



04. CNN 맛보기!

Computer Vision Tasks

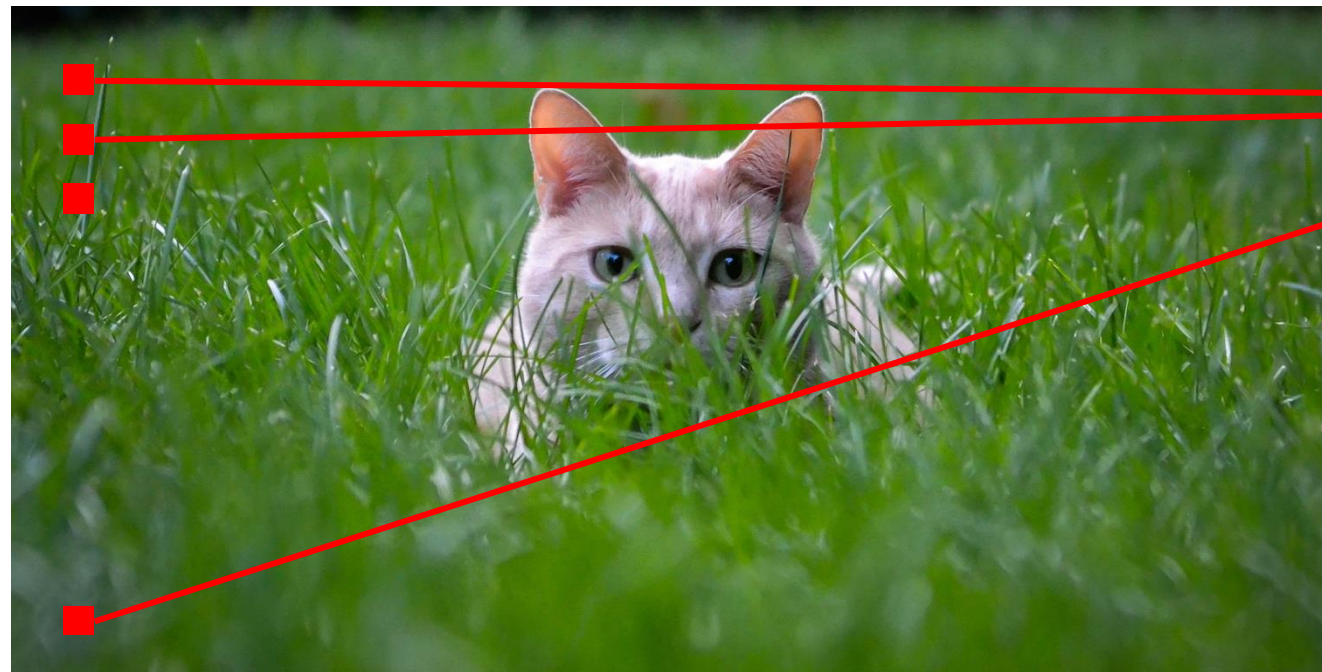


04. CNN 맛보기!

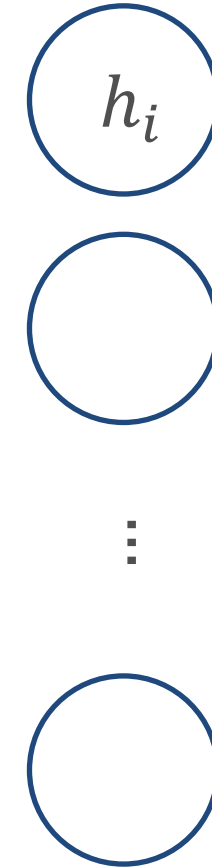
MLP on CV?

- 메모리 비효율적
- Optimization 어려움

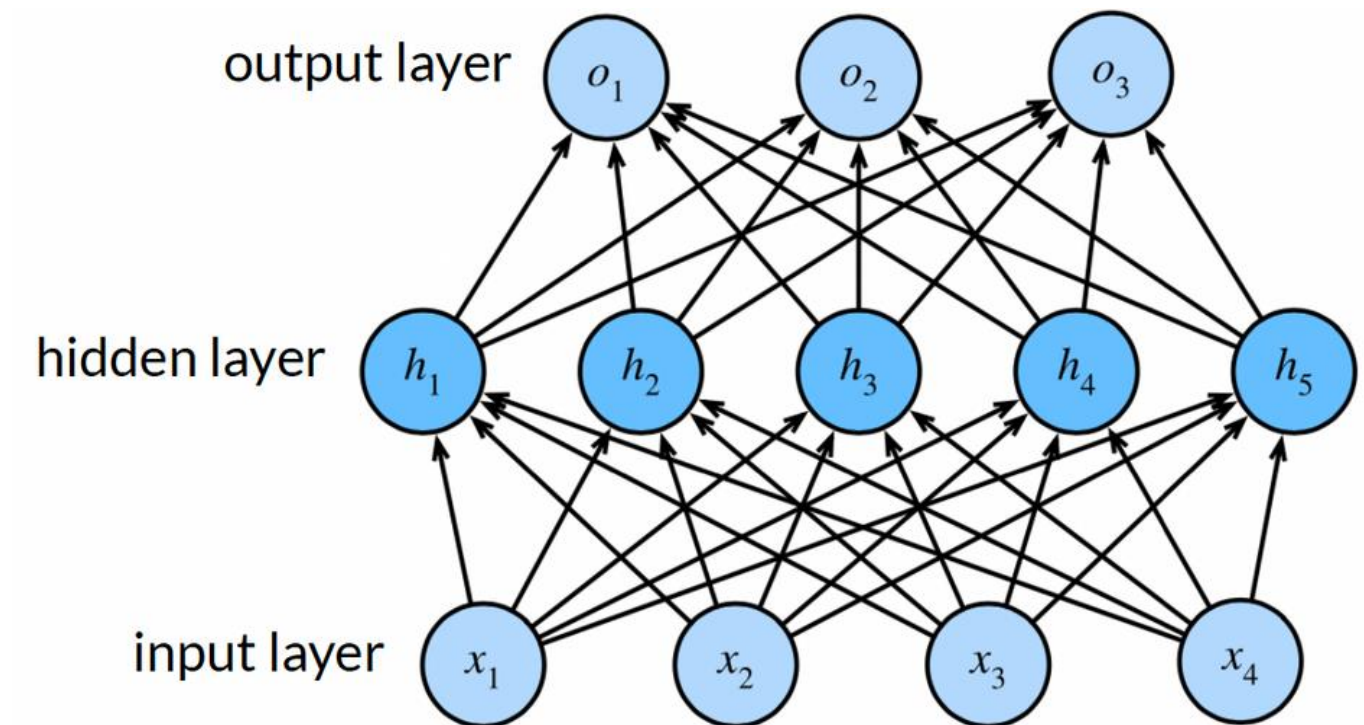
MLP에서는 모든 픽셀에 대해
각각 파라미터가 필요



x_j

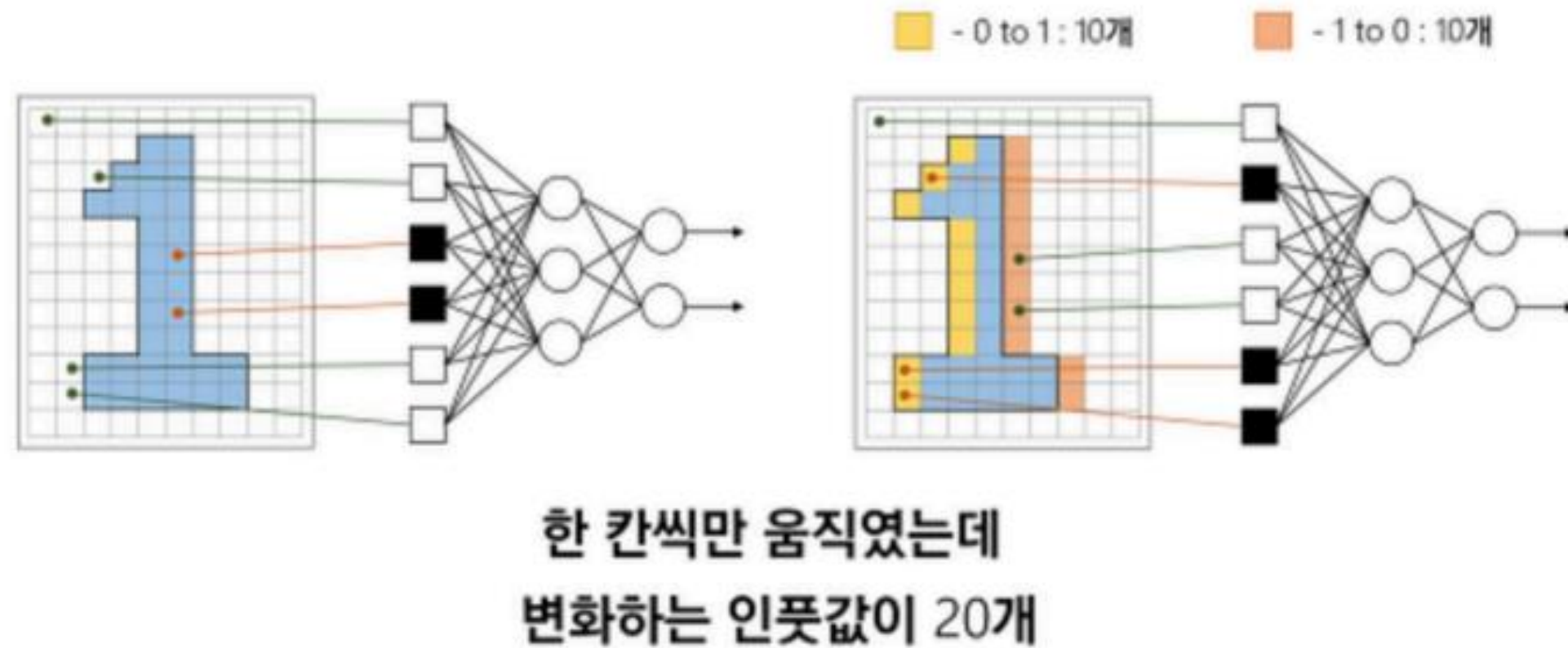


$$= \sum_j w_{ij} x_j$$



04. CNN 맛보기!

MLP on CV?

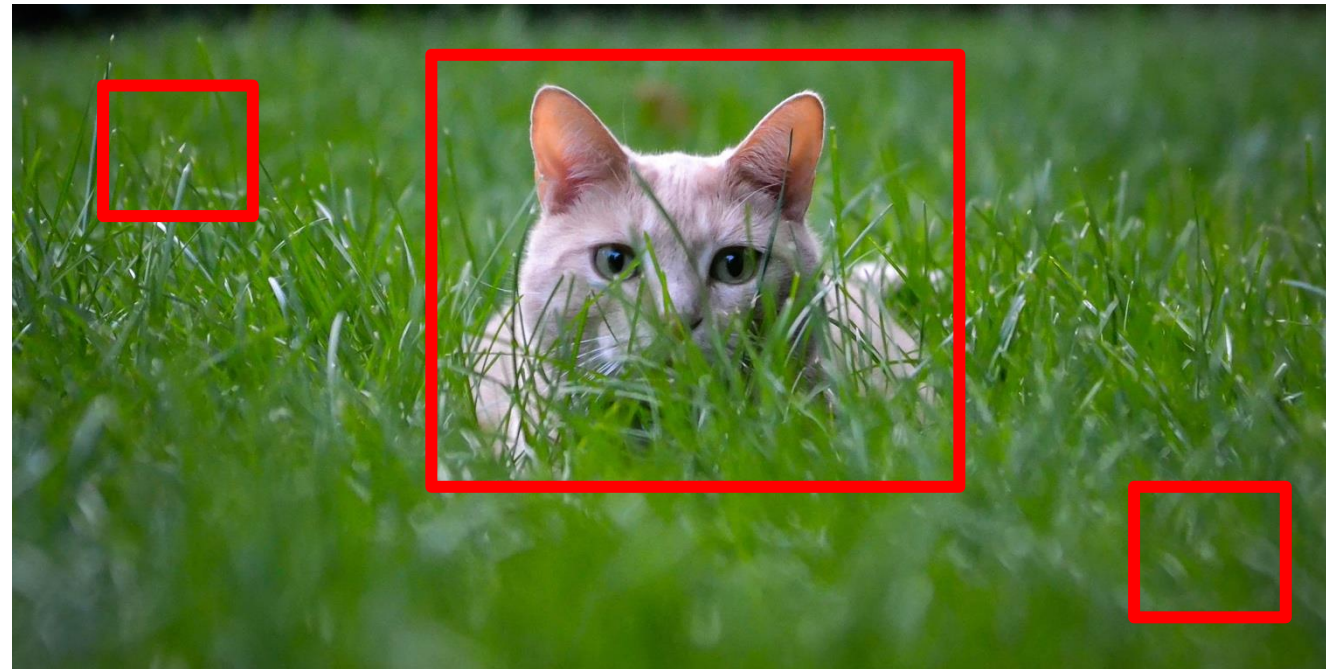


Why difficult?

이미지가 조금만 차이 나더라도 input의 변화가 매우 크게 발생함.

04. CNN 맛보기!

Principles for Visual Domain



모든 픽셀에 대한 정보를 활용하는 모델 (MLP)는
허위 상관(Spurious Correlation) 문제에 취약

- 이미지 데이터: 먼 지역의 내용(특징)은 무관

04. CNN 맛보기!

Principles for Visual Domain

→ Locality principle

먼 지역의 내용(특징)은 무관하므로,
이미지의 특정 부분을 인식할 때 그 부분과 멀리 떨어진 다른 지역의 정보는 중요하지 않다고 간주된다.

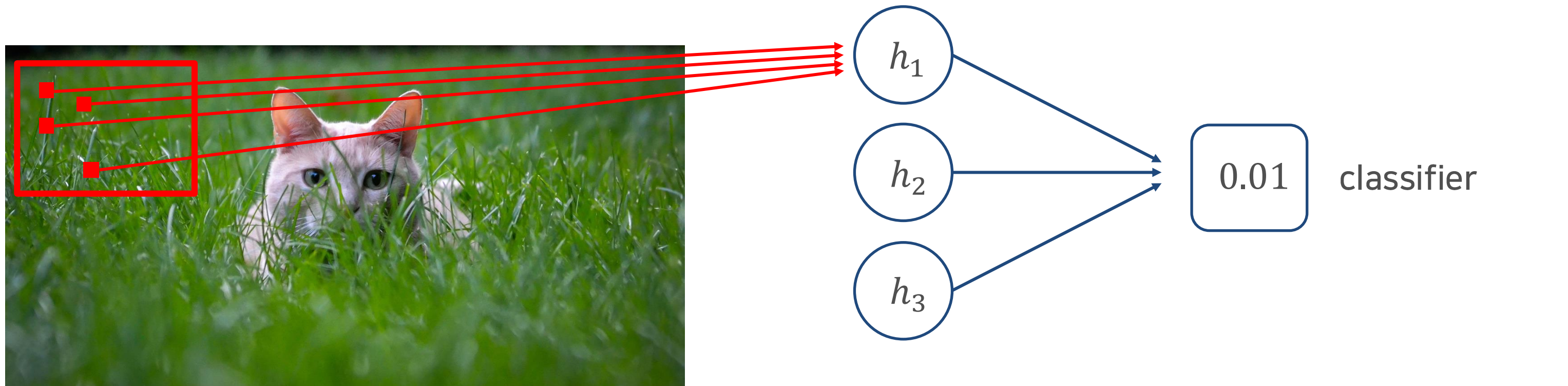
이미지를 스캔하면서 각 Image patch(조각)에 점수를 매긴다.
= 이미지의 구조는 국소적인 정보를 기반으로 처리될 수 있다.
= Local Info(특정 영역의 특징)을 활용한 효율적인 학습을 목표.

→ CNN(Convolutional Neural Network)은 Locality Principle에 기반하여 이미지를 처리!

04. CNN 맛보기!

Principles for Visual Domain

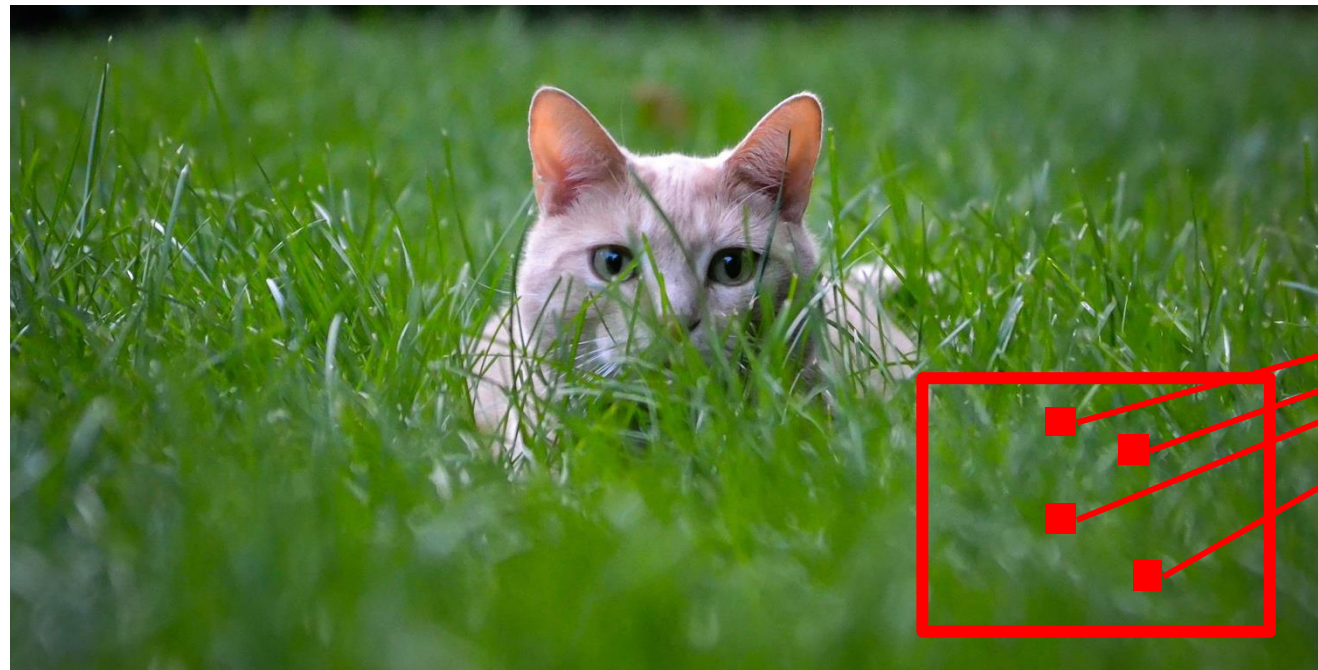
→ Locality principle



04. CNN 맛보기!

Principles for Visual Domain

→ Locality principle



h_1

h_2

h_3

이미지를 스위핑하여 각
패치(patch)에 점수 할당 가능

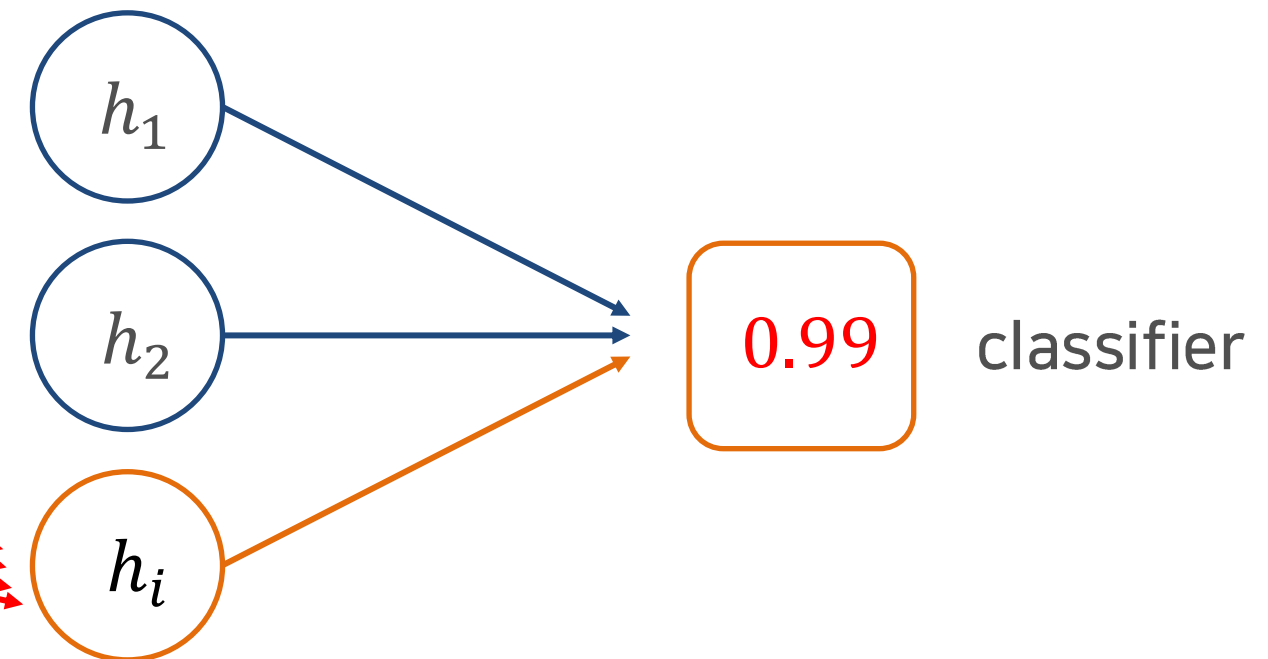
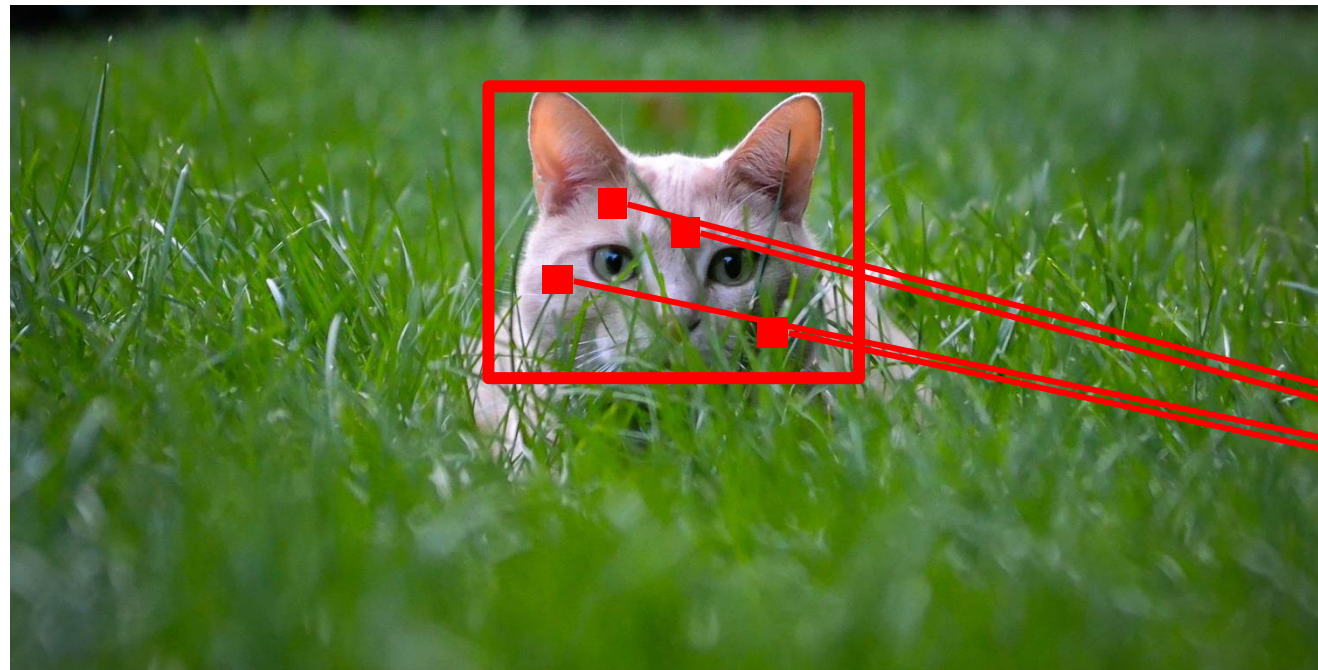
0.01

classifier

04. CNN 맛보기!

Principles for Visual Domain

→ Locality principle



04. CNN 맛보기!

Principles for Visual Domain

→ Locality principle

먼 지역의 내용(특징)은 무관하므로,
이미지의 특정 부분을 인식할 때 그 부분과 멀리 떨어진 다른 지역의 정보는 중요하지 않다고 간주된다.

→ Spatial Invariance

이미지 내 객체의 위치에 의존하지 않아야 한다.

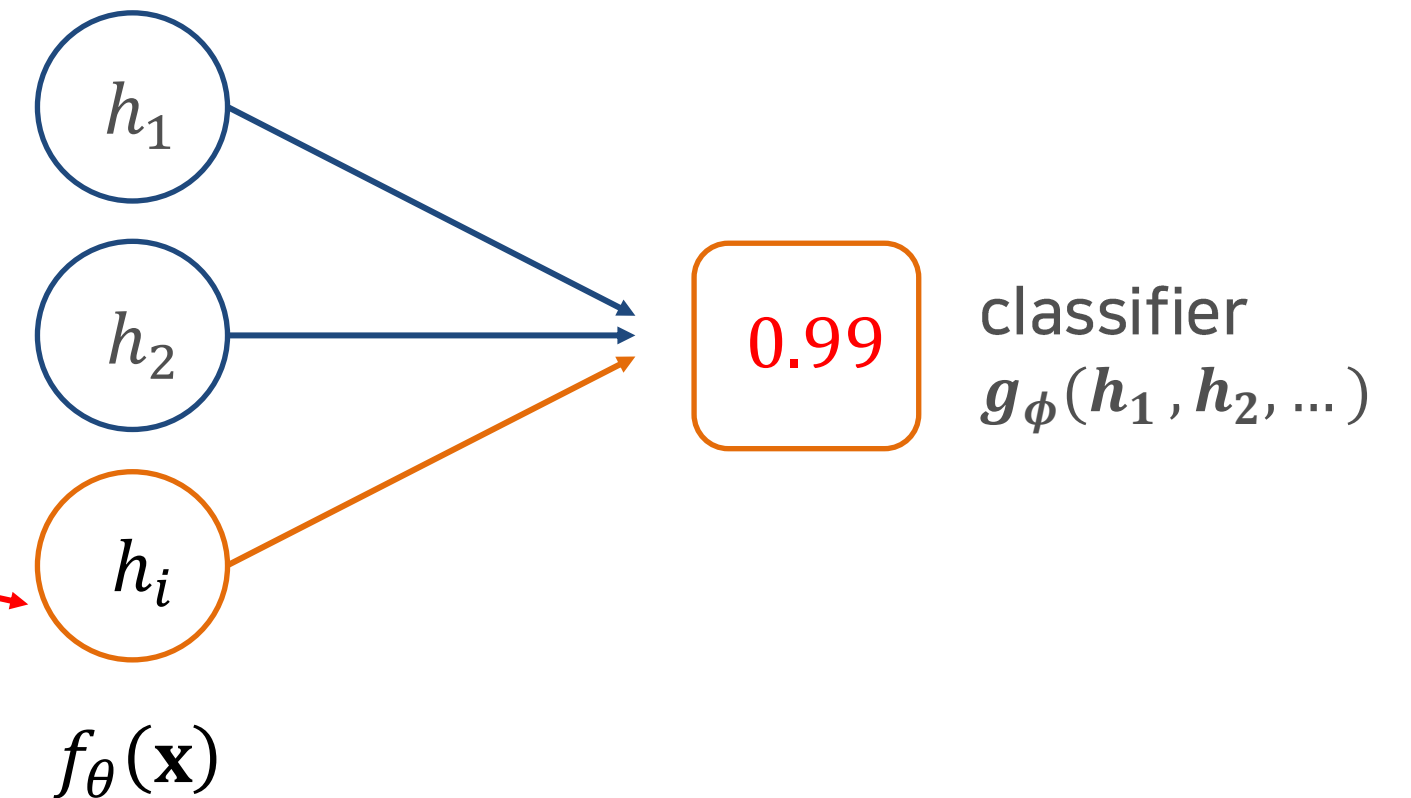
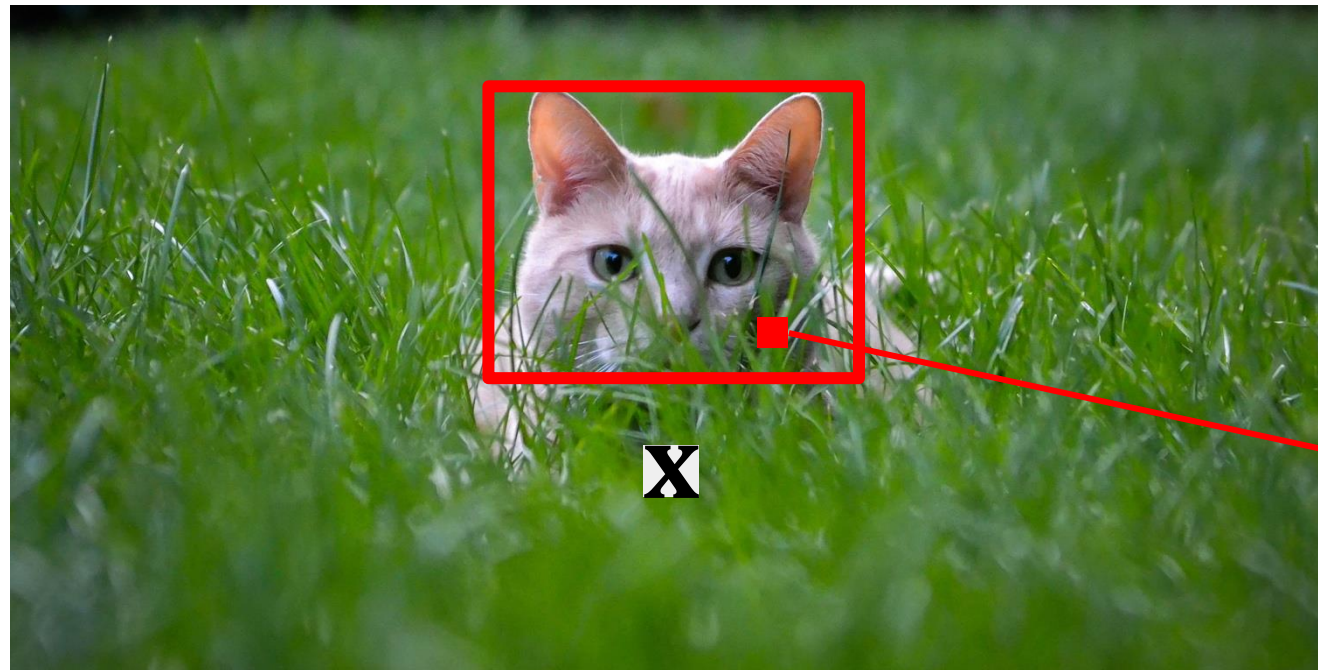
what object looks like does not depend upon *where the object is located*

Translation Equivariance & Translational Invariance

04. CNN 맛보기!

Principles for Visual Domain

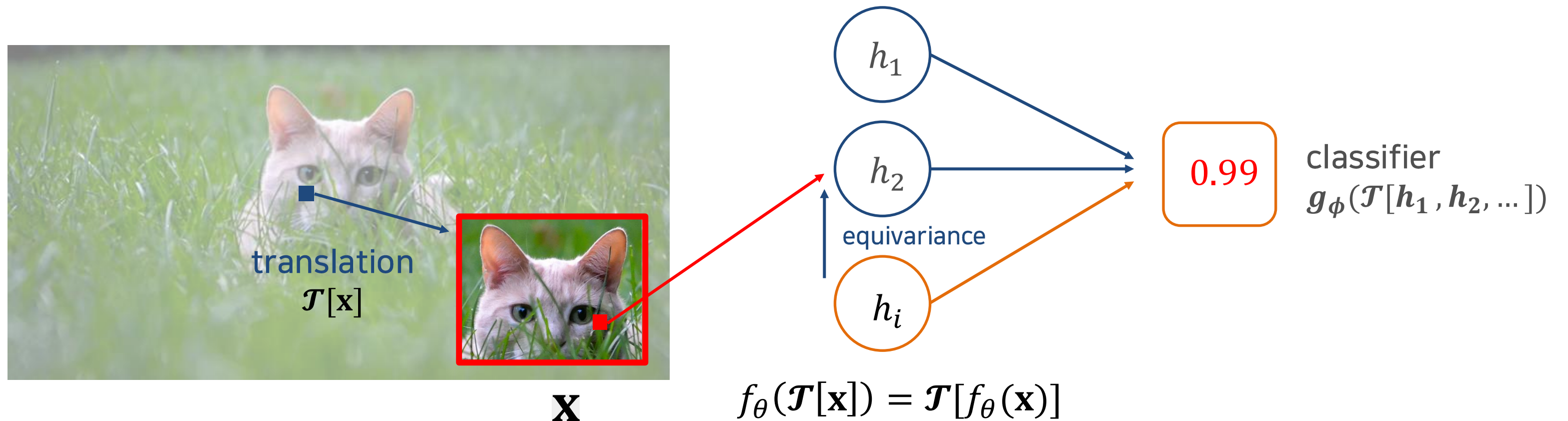
→ Locality principle



04. CNN 맛보기!

Principles for Visual Domain

→ Translational Equivariance



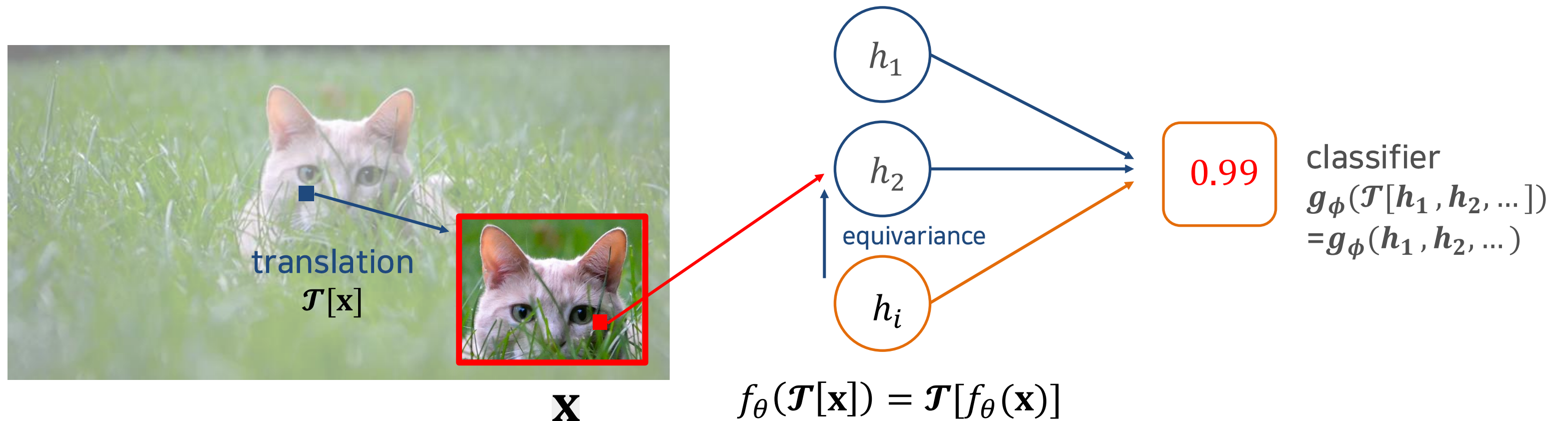
Translational Equivariance

: 객체가 이동(translate)하면, detector f_{θ} 는 **translate된 패치**에서 정보를 포착

04. CNN 맛보기!

Principles for Visual Domain

→ Translational Equivariance + Invariance



Translational Invariance

: 그에 따라 분류기 g_ϕ 는 translate된 패치의 정보를 이용해서 **동일한 결과**를 출력!

04. CNN 맛보기!

Principles for Visual Domain

**Semantic
Segmentation**



GRASS, CAT,
requires
equivariance

**Classification
+ Localization**



CAT
requires
invariance

**Object
Detection**



DOG, DOG, CAT
requires
equivariance

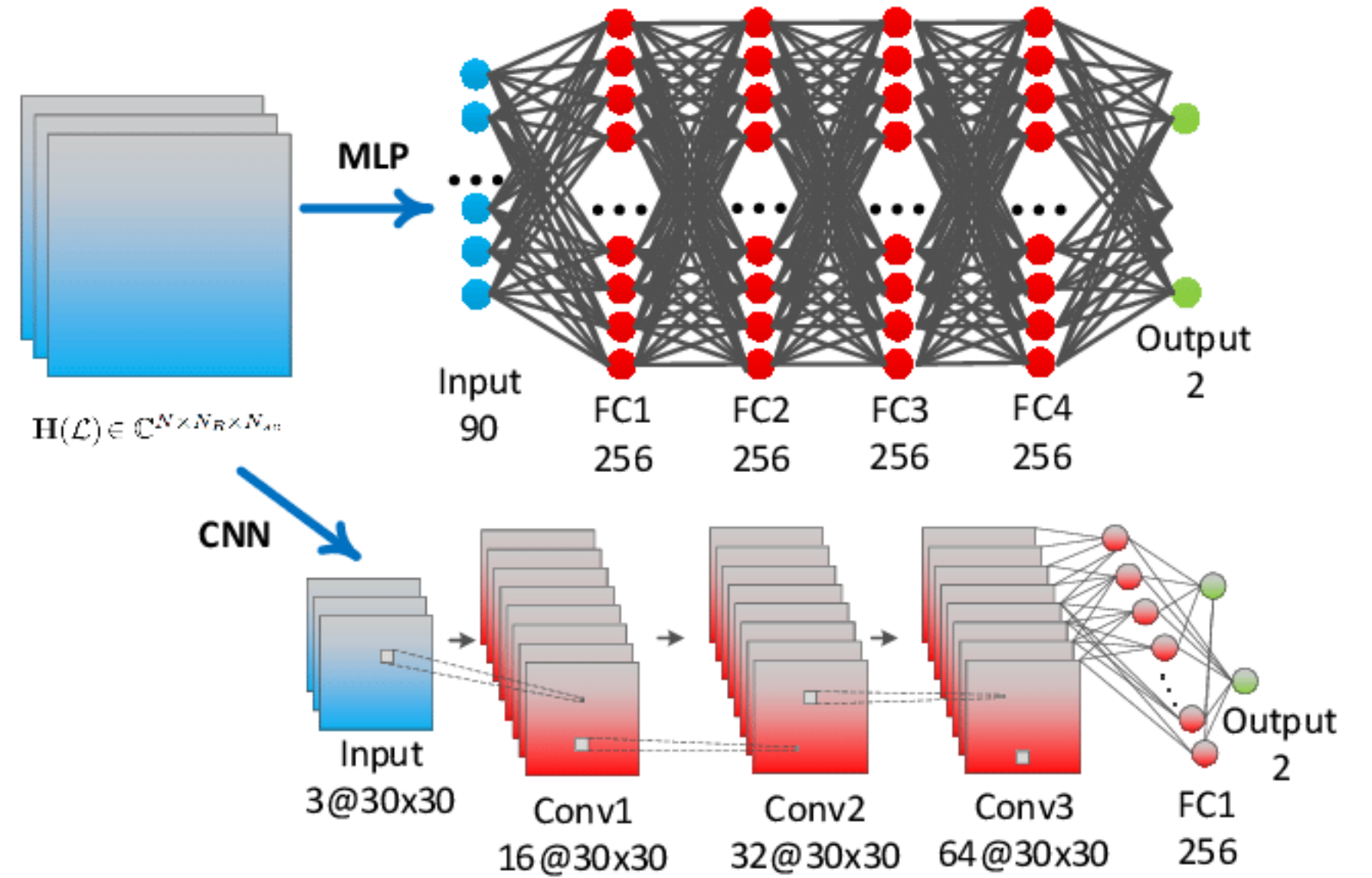
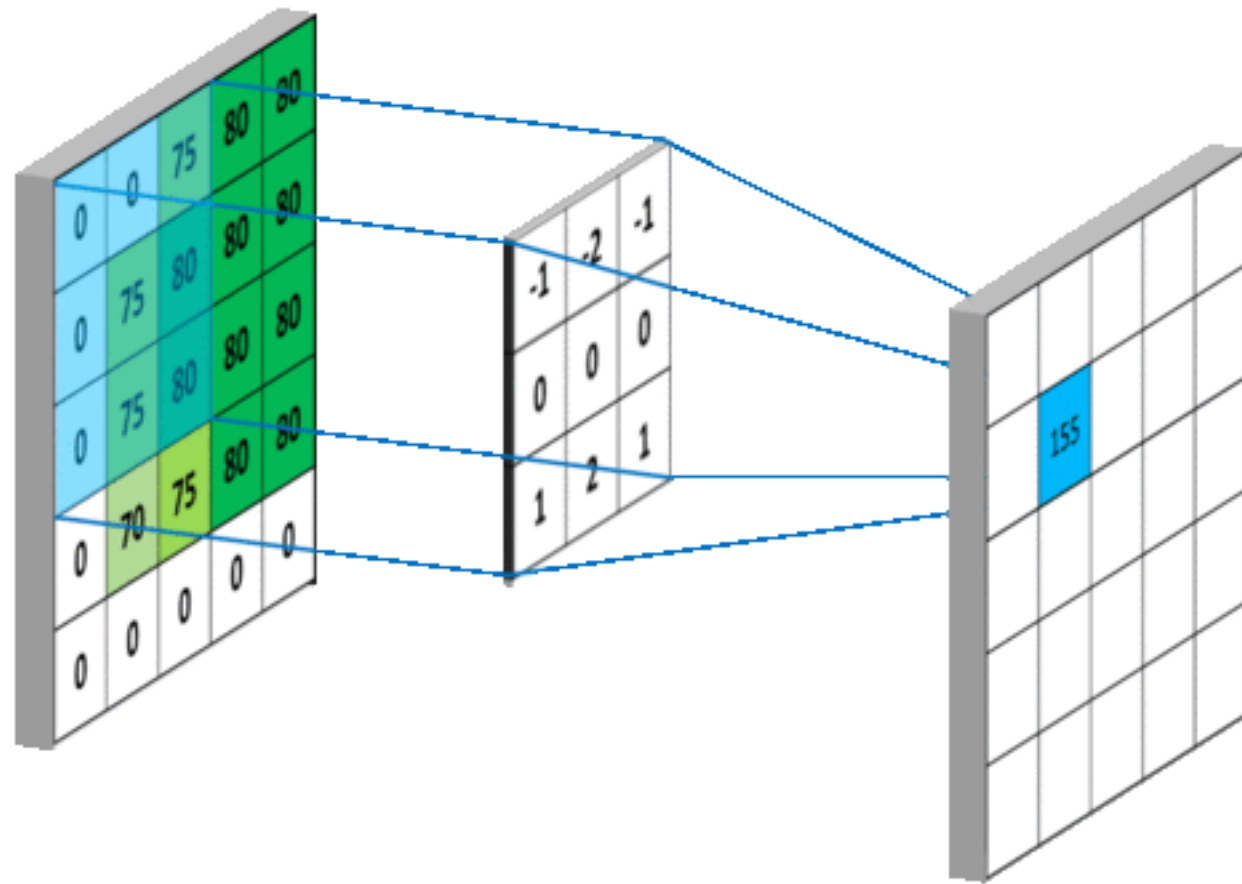
**Instance
Segmentation**



DOG, DOG, CAT
requires
equivariance

04. CNN 맛보기!

Solution: Convolution Operation



04. CNN 맛보기!

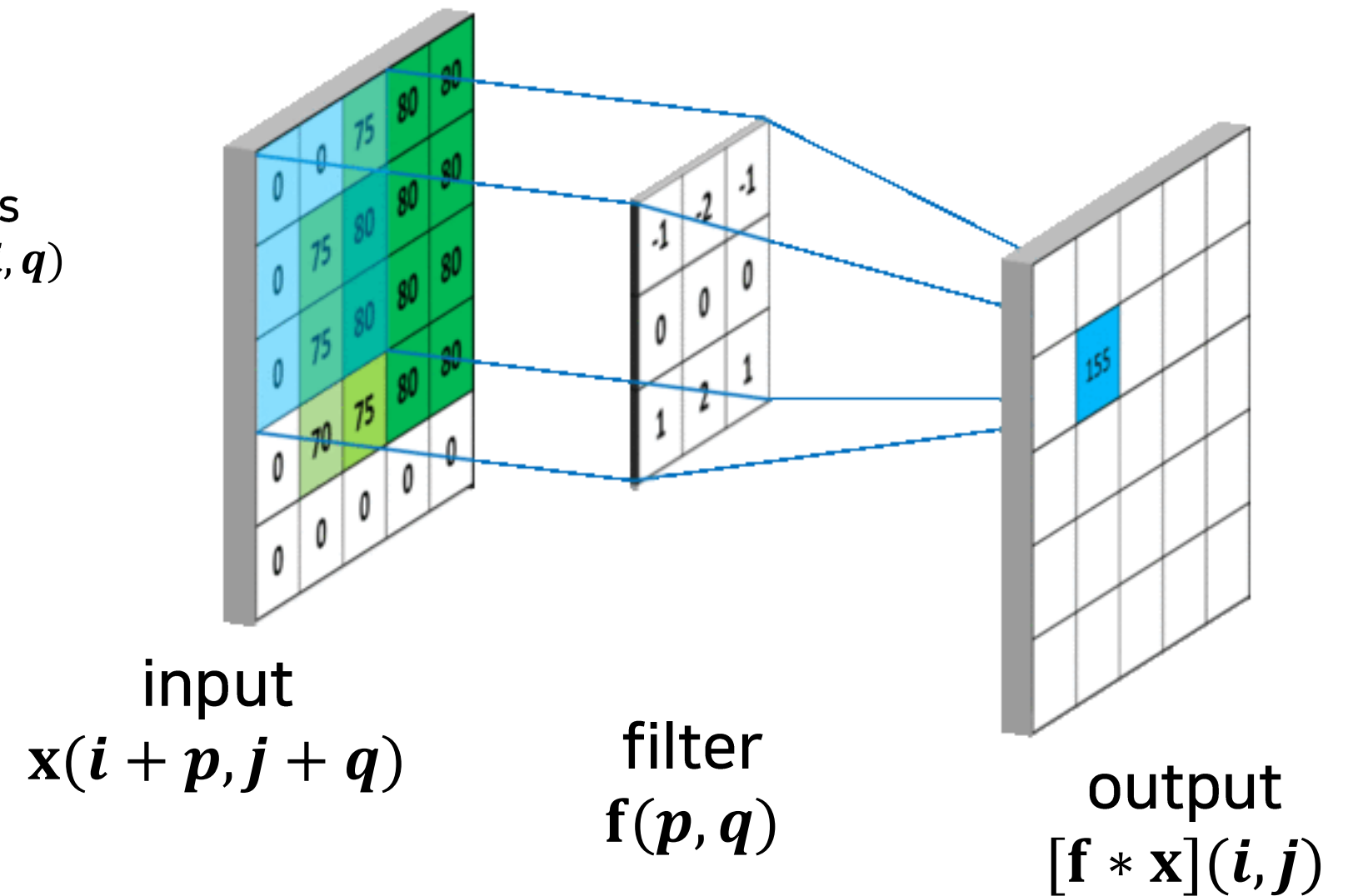
Solution: Convolution Operation

2D Conv

$$[\mathbf{f} * \mathbf{x}](i, j) = \sum_{p, q : |p|, |q| \leq \Delta} \mathbf{f}(p, q) \mathbf{x}(i + p, j + q)$$

patch size $\rightarrow$ locality principle

Convolution shares parameters $\mathbf{f}(\cdot)$ along the current locations (i, q)
 $\rightarrow$ spatial invariance



05

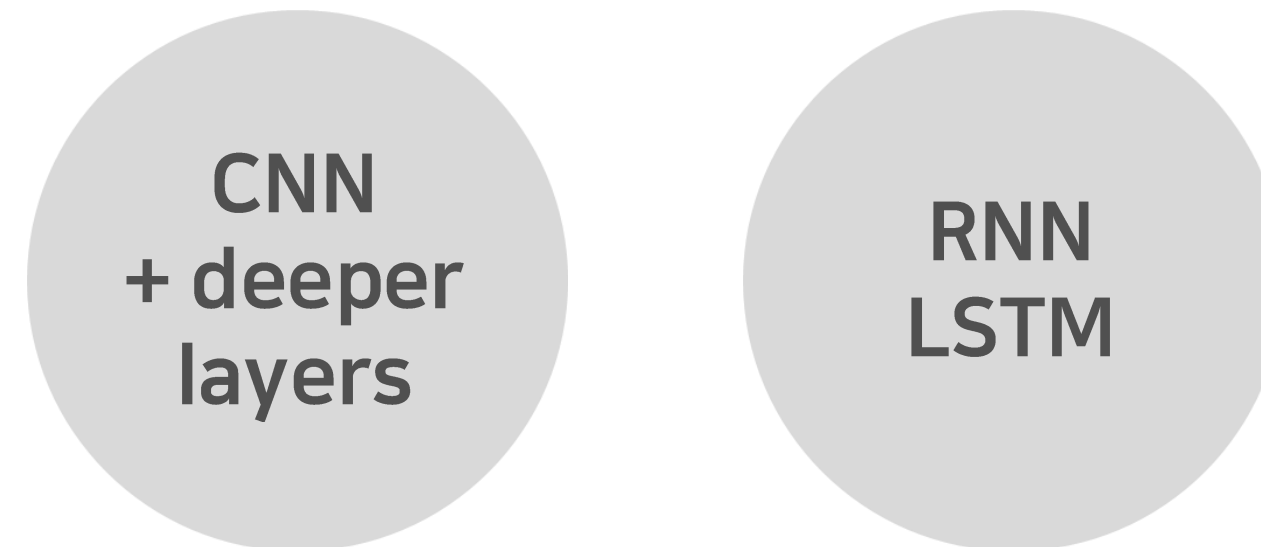
다음 주차 예고

05. 다음 주차 예고

4주차 Session Key Word



5주차 Session Key Word



05. 다음 주차 예고

4주차 과제

공부 과제

d2l
6장
CNN

오승상 딥러닝
16, 17, 18강

코드 과제

슬랙 공지로
추후 전달 예정

05. 다음 주차 예고

KUBIG CONTEST 관련 공지

2월 27일 금요일 contest 진행

아래 주제는 참고사항이며, 원하는 주제로 진행해도 됩니다!

6주차 (2월 12일)에 중간발표가 있을 예정입니다 ☺

(프로젝트 주제 및 현황에 대해 간단히 발표 준비해주세요)

*유의 : github 코드 제출 및 발표 ppt 제출 필수

<지난 기수 DL Session 주제>

난독화된 한글 리뷰 복원(NLP)

부동산 허위매물 분류 (Tabular)

저해상도 조류 사진 분류 (CV, Detection-Classification)

지원 고객 등급 분류 (Tabular)

Thank You

2026 Winter DL Session 4주차 끝!



고려대학교
KOREA UNIVERSITY

KUBIG
DATA SCIENCE & AI

Copyright © 2026
by KUBIG